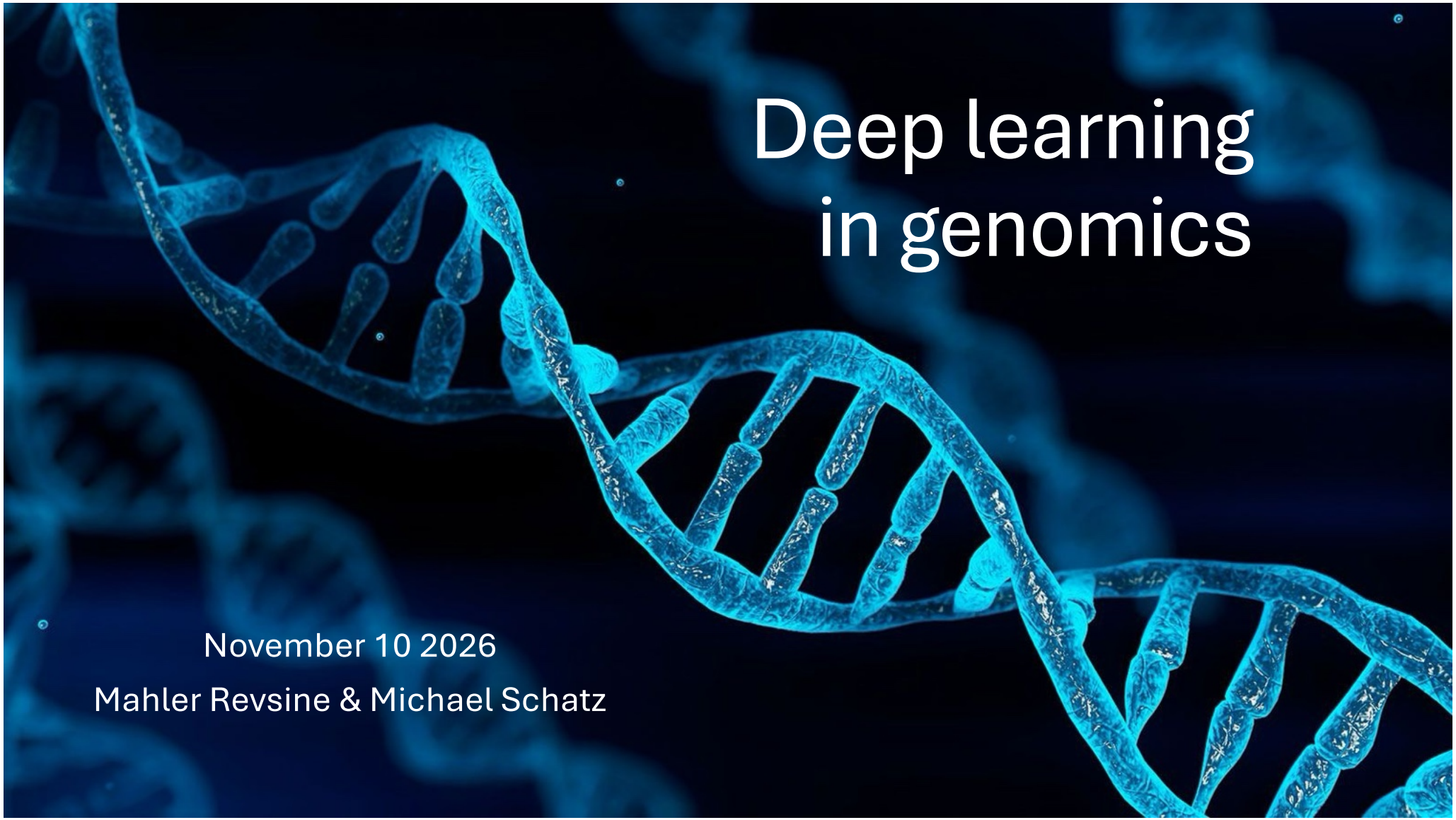


Deep learning in genomics

November 10 2026

Mahler Revsine & Michael Schatz

Announcements

- Preliminary project report due tonight
- Assignment 5 due Sunday night
  - Must be in by start of class Monday!

Recap

“Transformers”

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.



Ashish Vaswani
Essential AI



Llion Jones
Sakana AI



Noam Shazeer
Character AI



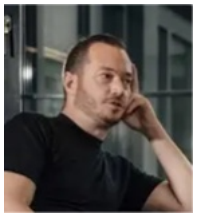
Aidan Gomez
Cohere



Niki Parmar
Essential AI



Łukasz Kaiser
OpenAI



Jakob Uszkoreit
Inceptive



Illia Polosukhin
NEAR

A Sequence Modeling Problem: Predict the Next Word

f("This morning I took my cat for a) = ???

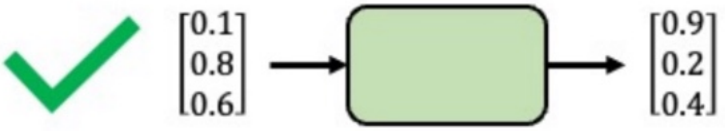
given these words

predict the next word

Representing Language to a Neural Network



Neural networks cannot interpret words



Neural networks require numerical inputs

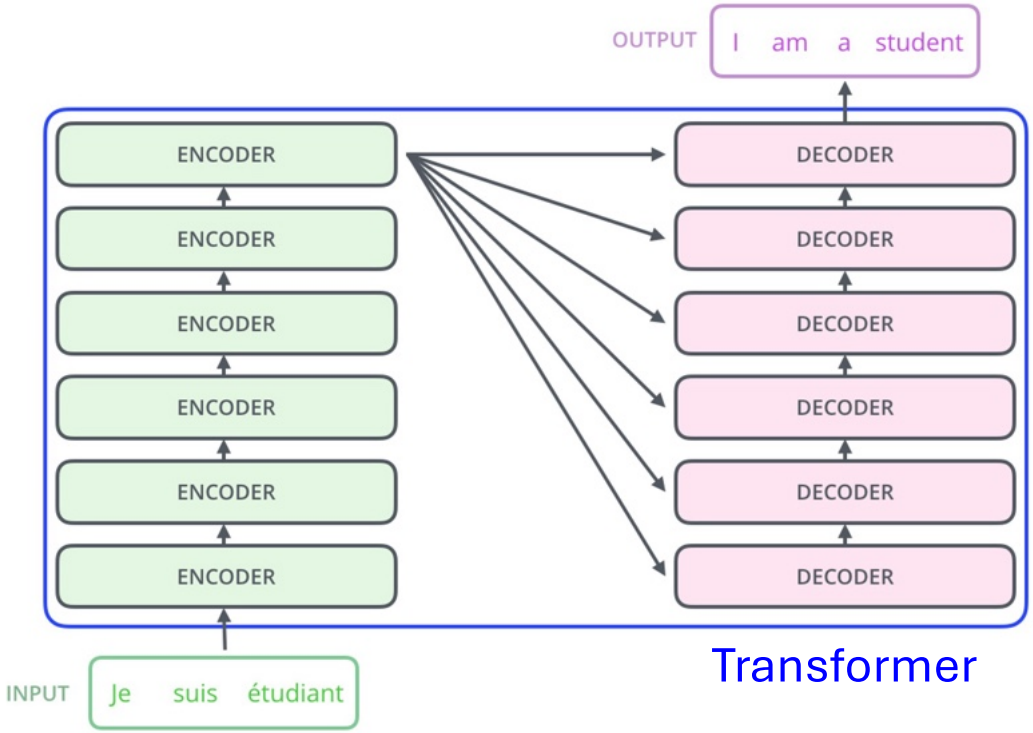
The application



Trained with a huge collection of pairs of sentences in french & english
Measure accuracy using “BLEU” scores to gold standard (higher is better)
Use a “reasonable” amount of GPU time for training (<1 week on 8 GPUs)

https://jalammar.github.io/illustrated-transformer/

The architecture



Transformer

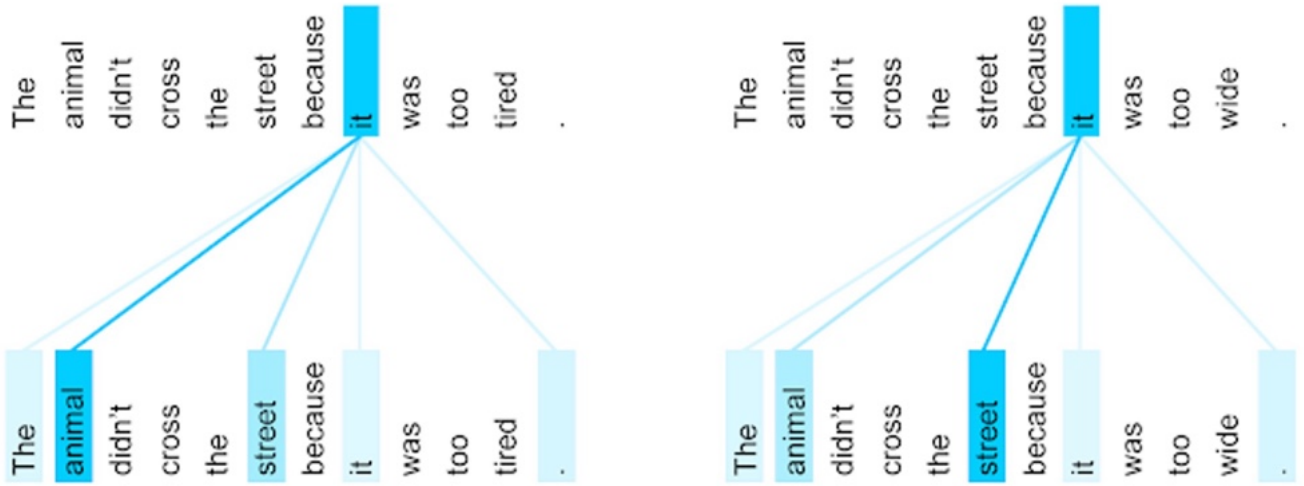
Encoder transforms sentences in language A into a highly abstract embedded space
Decoders progressively decodes highly abstract embedded space into language B

https://jalammar.github.io/illustrated-transformer/

Coreference resolution: self attention

The animal didn't cross the street because it was too tired.
L'animal n'a pas traversé la rue parce qu'il était trop fatigué.

The animal didn't cross the street because it was too wide.
L'animal n'a pas traversé la rue parce qu'elle était trop large.



https://research.google/blog/transformer-a-novel-neural-network-architecture-for-language-understanding/

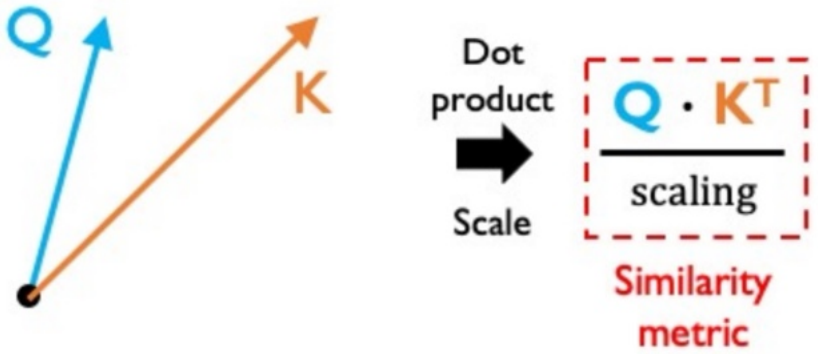
Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute **attention weighting**
4. Extract features with high attention

Attention score: compute pairwise similarity between each query and key

How to compute similarity between two sets of features?



Also known as the "cosine similarity"

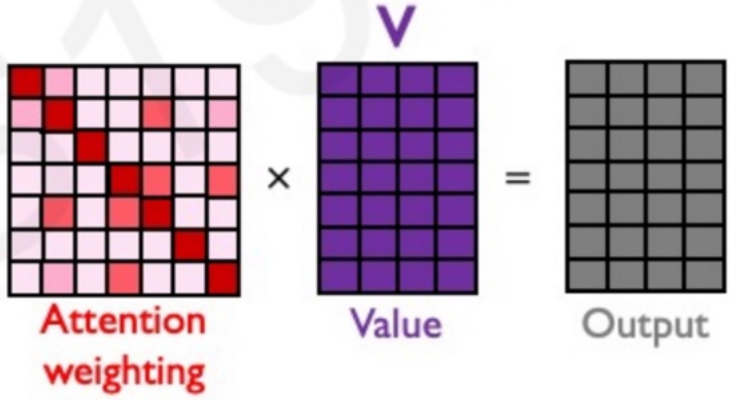
MIT 6.S191

Learning Self-Attention with Neural Networks

Goal: identify and attend to most important features in input.

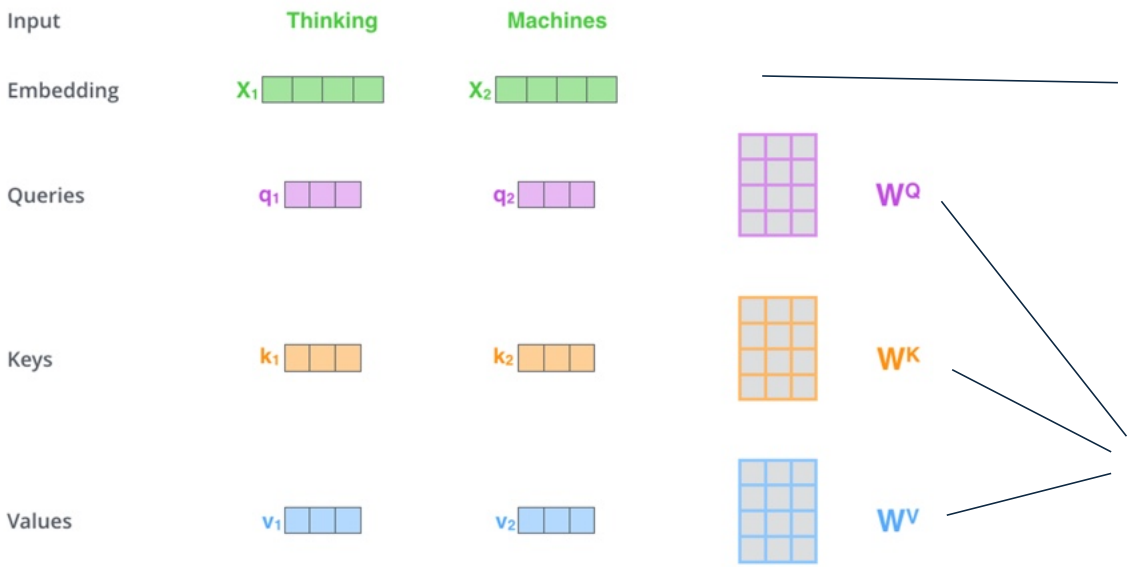
1. Encode **position** information
2. Extract **query, key, value** for search
3. Compute **attention weighting**
4. Extract **features with high attention**

Last step: self-attend to extract features



$$softmax\left(\frac{Q \cdot K^T}{scaling}\right) \cdot V = A(Q, K, V)$$

MIT 6.S191



Embeddings are learned to project similar words into similar directions (cat & dog; king & queen)

- Similar to AlexNet!

Weight matrices (“masks”) learned during training

Multiply embedded input value times WQ, WK, and WV to compute q, k, v

Query (Q): Represents the current token's intention to "seek information" from other tokens in the sequence.

- It asks the question, "Which parts of the input sequence are relevant to me?"

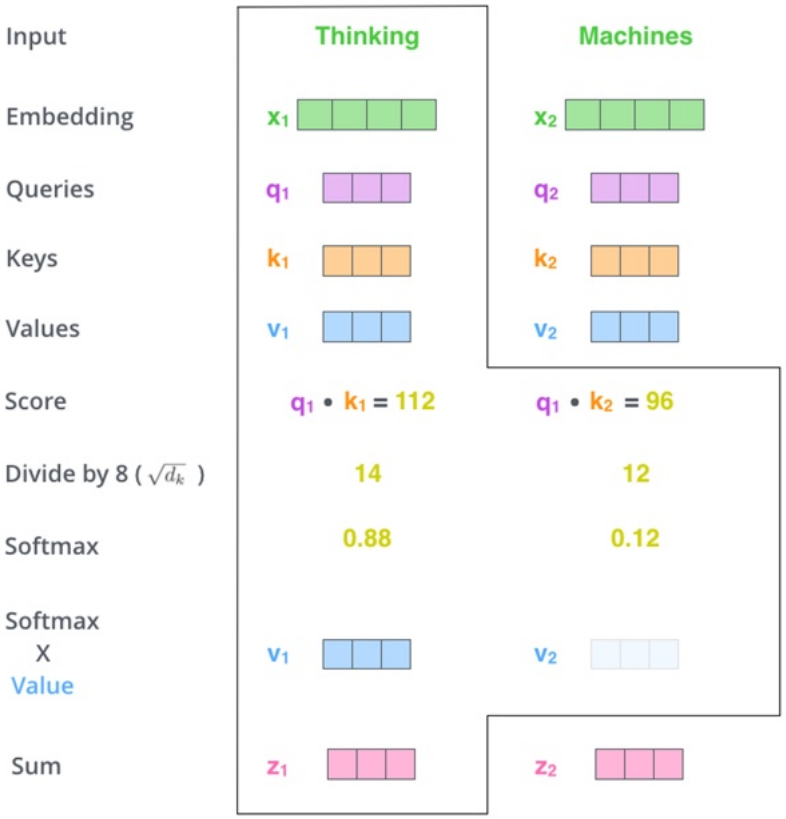
Key (K): Represents the content of the tokens against which the queries are compared.

- It acts like an "address" or "identifier" that the query uses to find relevant information.

Value (V): Contains the actual information that will be attended to. Once the attention mechanism determines which keys are most relevant to the query, the corresponding values are aggregated to form the final output.

https://jalammar.github.io/illustrated-transformer/

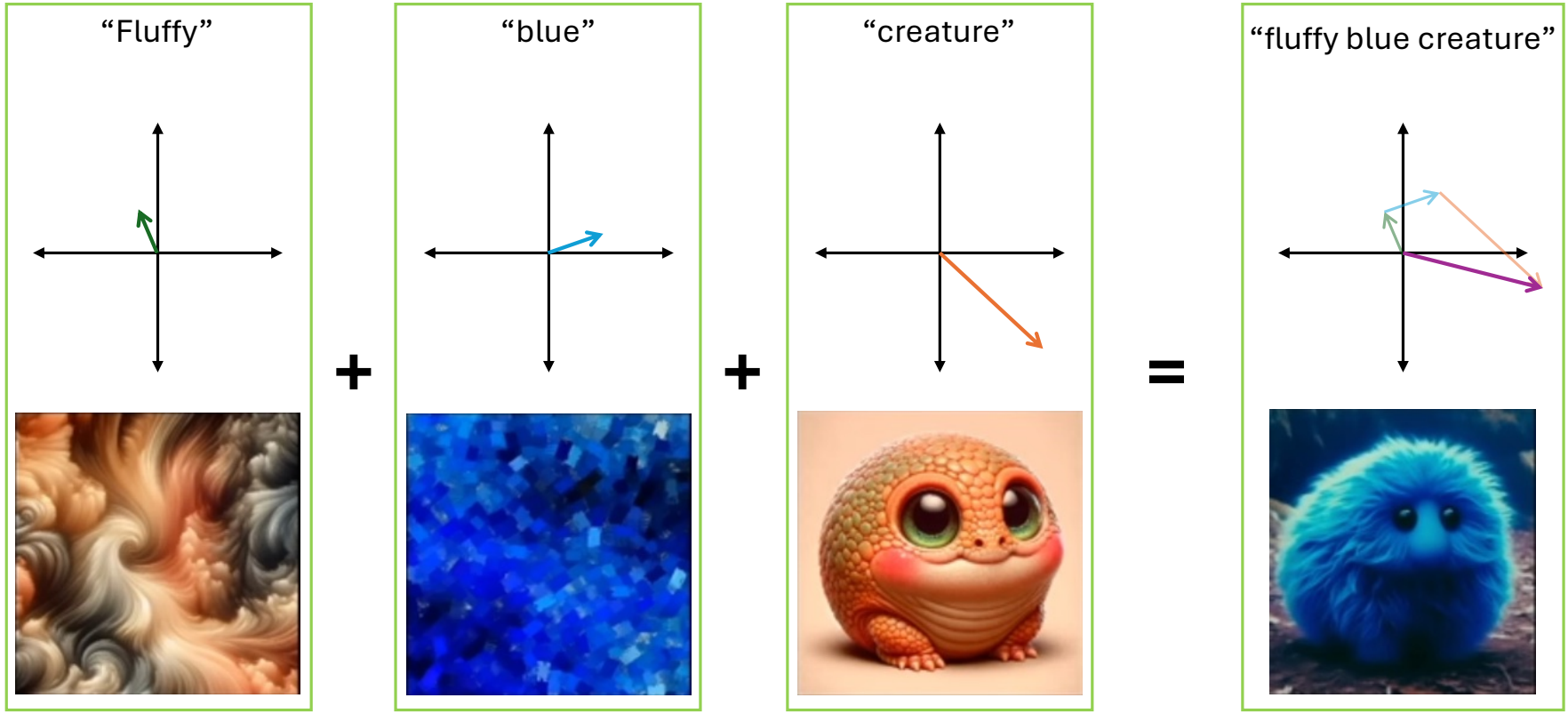
Attention computation

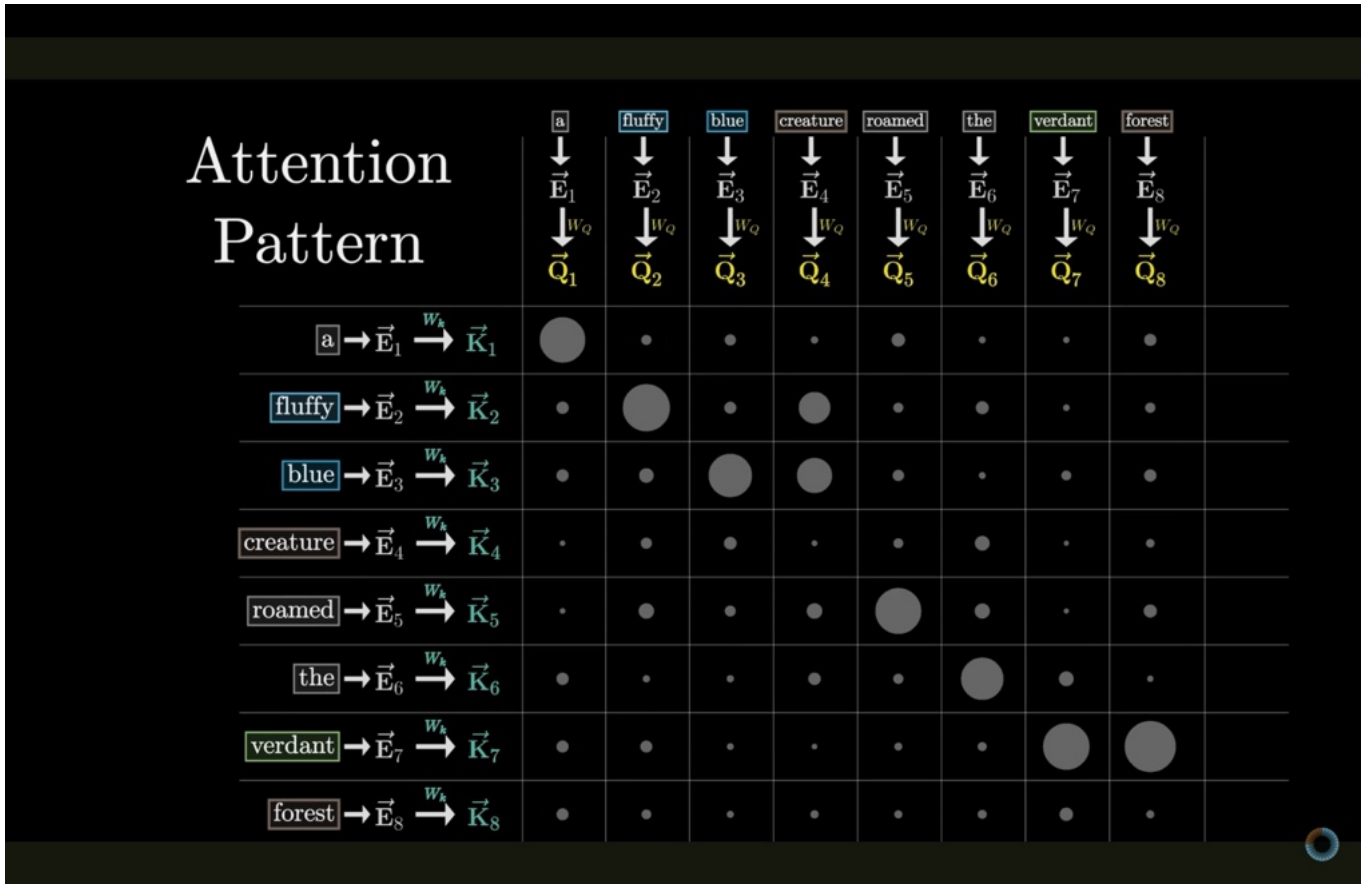


1. The score is calculated by taking the dot product (cosine similarity) of the **query vector** with the **key vector** of the respective word we're scoring.
2. The next steps are to divide the **scores** by 8 (the square root of the dimension of the key vectors used in the paper – 64). This leads to having more stable gradients.
3. Softmax normalizes the **scores** so they're all positive and add up to 1. This softmax **score** determines how much each word will be expressed at this position. ("Thinking" is mostly "Thinking" but a little "Machines")
4. Multiply each value vector by the softmax score to keep values of the word(s) we want to **focus** on, and drown-out irrelevant words
5. Sum up the weighted value vectors to produces the **output** of the self-attention layer at this position (for the first word). "Thinking" becomes "Thinking-machines"

https://jalammar.github.io/illustrated-transformer/

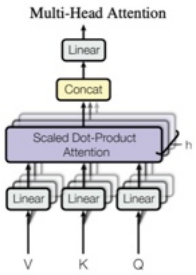
Attention **modifies** the **vector representation** of a **word** to include its **additional meaning**





Attention tells us which words **relate** to which other words

Multi-headed Attention



1) This is our input sentence*

2) We embed each word*

3) Split into 8 heads. We multiply X or R with weight matrices

4) Calculate attention using the resulting Q/K/V matrices

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

Thinking Machines

X

W_0^Q W_0^K W_0^V

Q_0 K_0 V_0

Z_0

W_1^Q W_1^K W_1^V

Q_1 K_1 V_1

Z_1

...

W_7^Q W_7^K W_7^V

Q_7 K_7 V_7

Z_7

W^O

Z

* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

R

Similar to having multiple convolutional filters

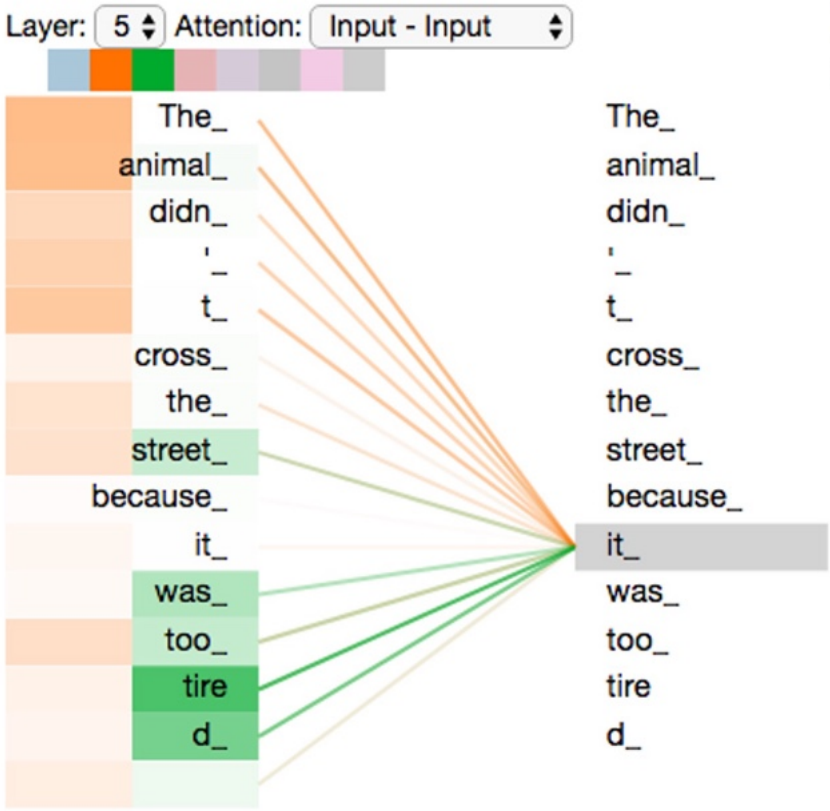
https://jalammar.github.io/illustrated-transformer/

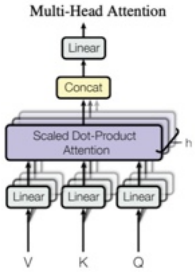
Multi-headed Attention

The orange attention head “knows” “it” refers to animal

The green attention head “knows” “it” refers to tired

Added together we form a new vector representing the concept of “tired animal”





https://jalammar.github.io/illustrated-transformer/

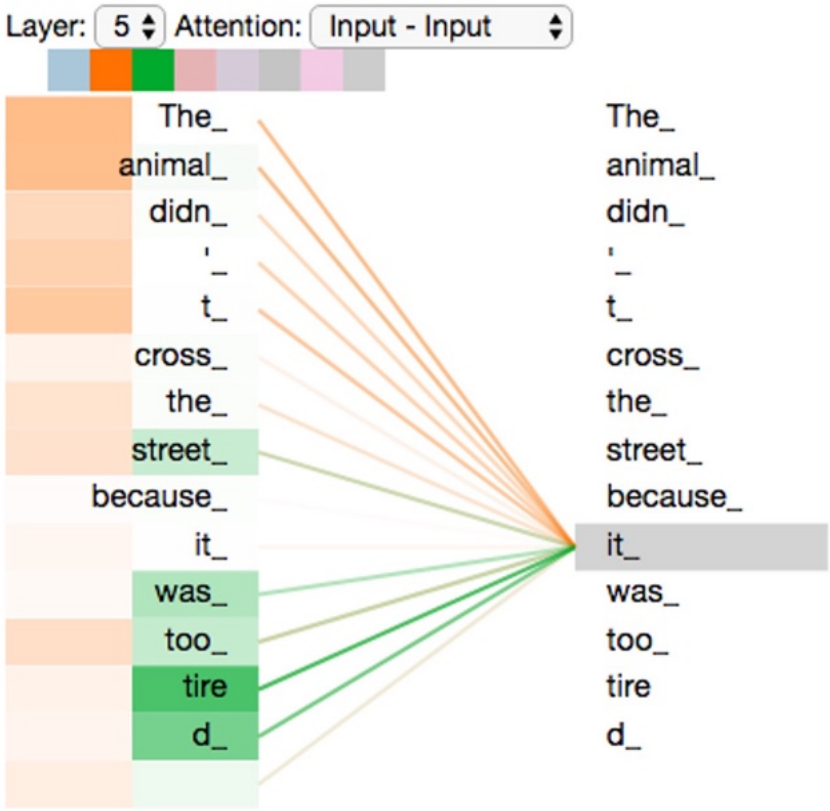
Multi-headed Attention

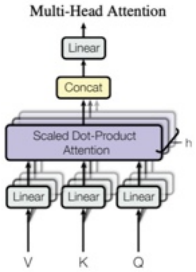
The orange attention head “knows” “it” refers to animal

The green attention head “knows” “it” refers to tired

Added together we form a new vector representing the concept of “tired animal”

* One head will be trained to look at previous word





https://jalammar.github.io/illustrated-transformer/

Transformer: Multi-headed attention to encode and decode

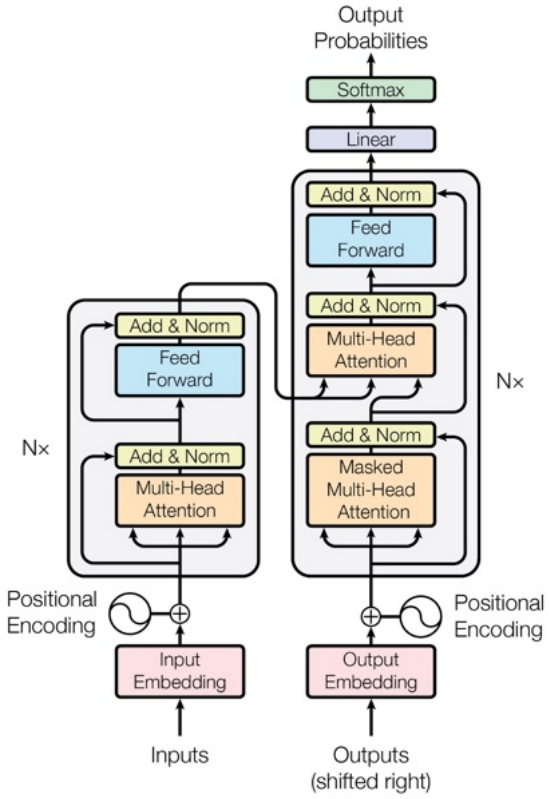


Figure 1: The Transformer - model architecture.

The Transformer uses multi-head attention in three different ways:

"Encoder self-attention": In a self-attention layer all of the keys, values and queries come from the same place, in this case, the output of the previous layer in the encoder. Each position in the encoder can attend to all positions in the previous layer of the encoder.

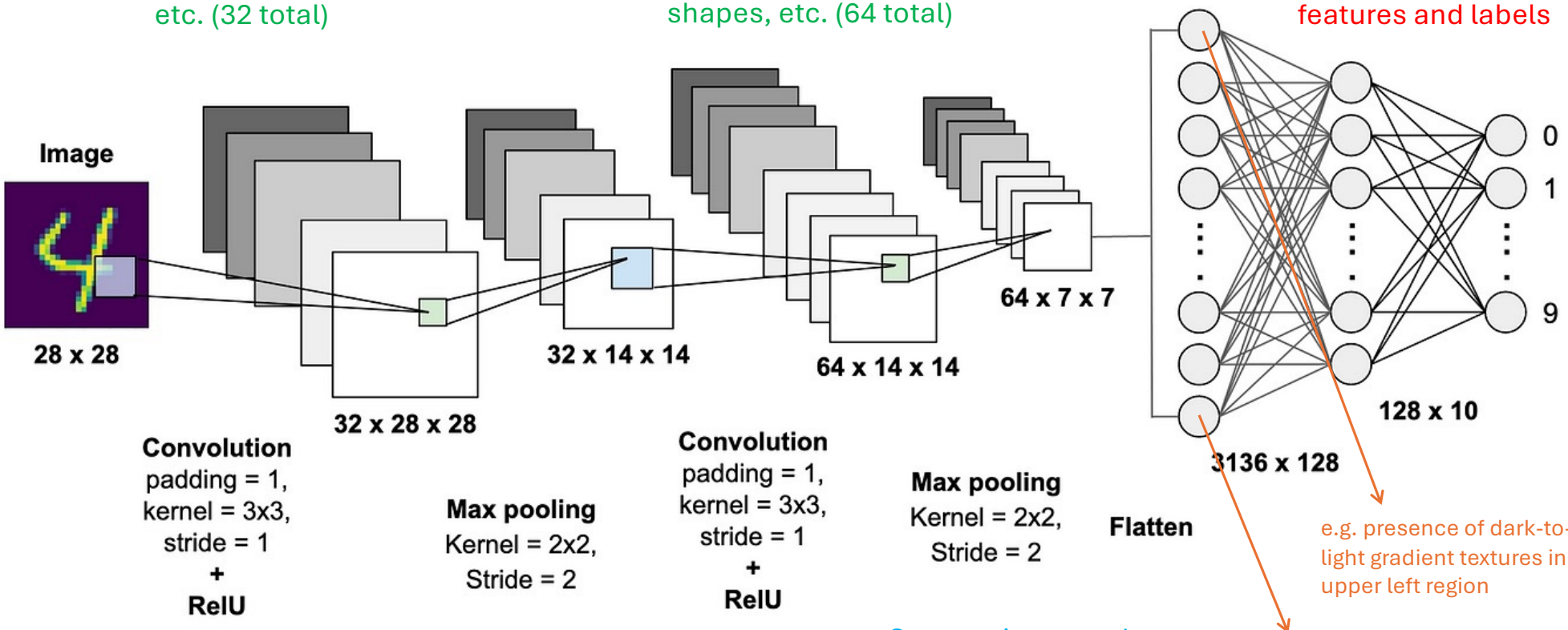
"Encoder-decoder attention": The queries come from the previous decoder layer, and the memory keys and values come from the output of the encoder. This allows every position in the decoder to attend over all positions in the input sequence.

"Self-attention in the decoder": Allows each position in the decoder to attend to all positions in the decoder up to and including that position. We need to prevent leftward information flow in the decoder to preserve the auto-regressive property. We implement this inside of scaled dot-product attention by masking out (setting to $-\infty$) all values in the input of the softmax which correspond to illegal connections.

In each 3x3 region: Simple features e.g. vertical lines, horizontal lines, etc. (32 total)

In each 8x8 region: Complex features e.g. patterns, textures, combinations of shapes, etc. (64 total)

Learn complex patterns between features and labels



Summarize simple features in each 4x4 region

Summarize complex features in each 10x10 region

e.g. presence of dark-to-light gradient textures in upper left region

e.g. presence of trident shapes in bottom right region

Decoding

Magic! Joint attention between abstracted input language to abstracted output language

Self-attention of input stream

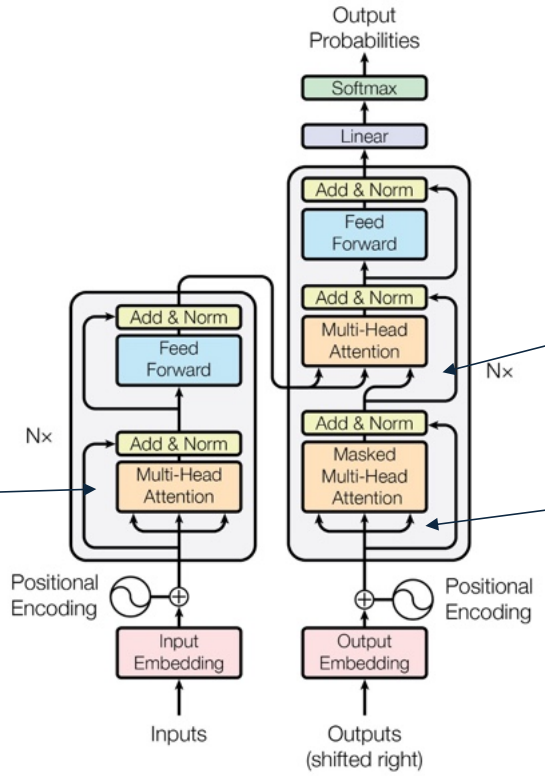


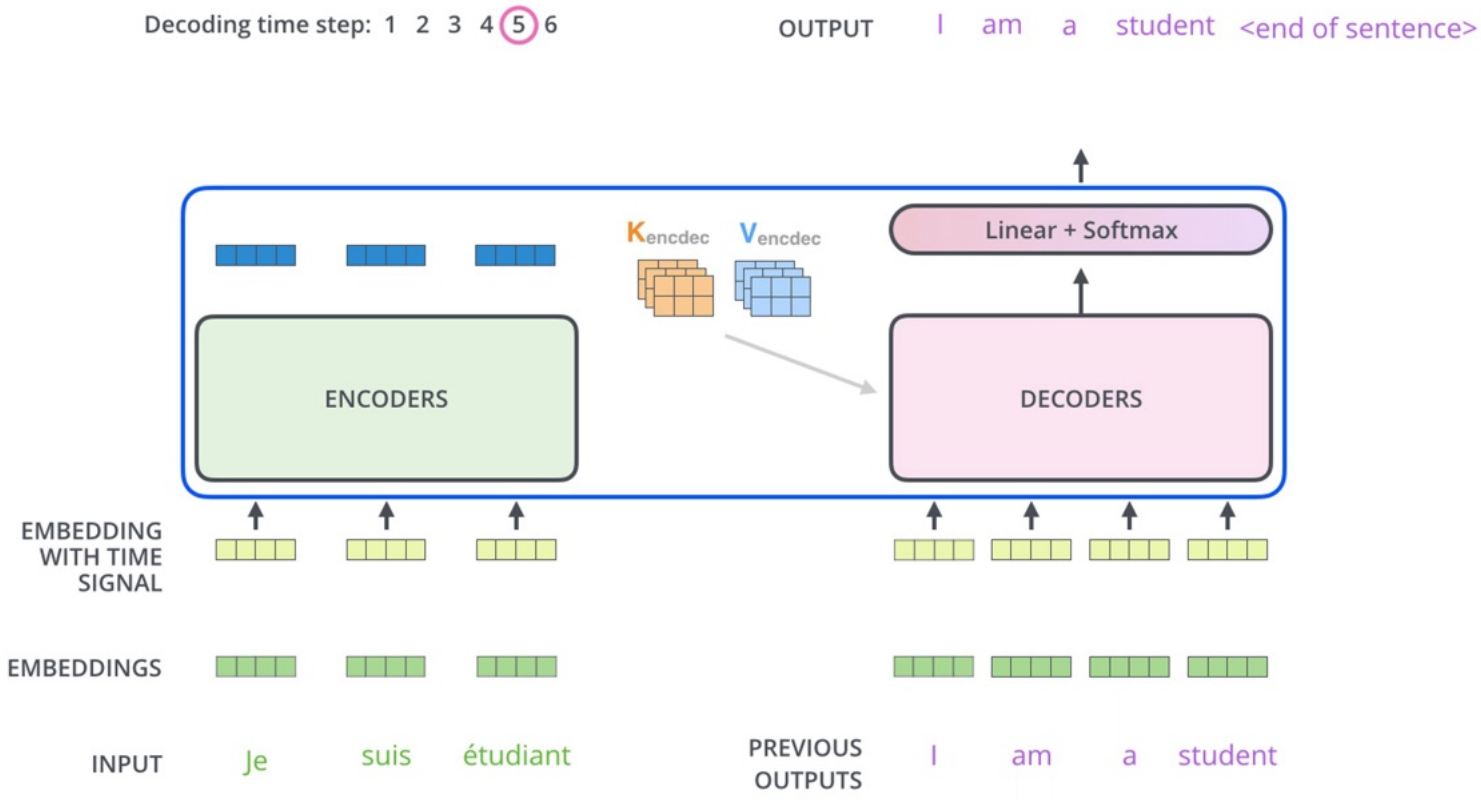
Figure 1: The Transformer - model architecture.

Self-attention of output stream

Encoder transforms sentences in language A into a highly abstract embedded space
Decoders progressively decodes highly abstract embedded space into language B

https://jalammar.github.io/illustrated-transformer/

Decoding



https://jalammar.github.io/illustrated-transformer/

Performance

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	**$3.3 \cdot 10^{18}$**	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

More accurate and 100 to 1000 times faster!

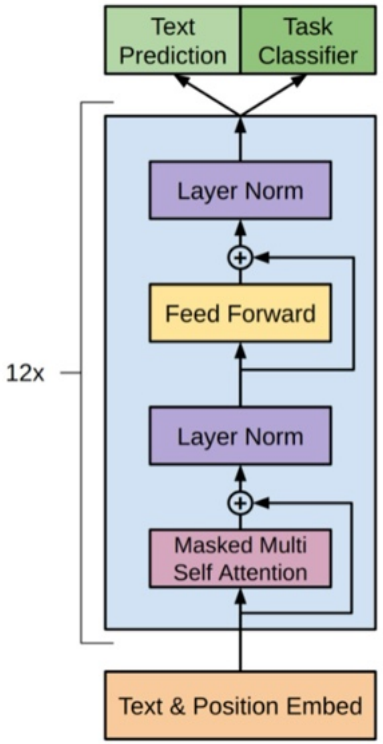
GPT-1

Improving Language Understanding by Generative Pre-Training

Alec Radford OpenAI alec@openai.com
Karthik Narasimhan OpenAI karthikn@openai.com
Tim Salimans OpenAI tim@openai.com
Ilya Sutskever OpenAI ilyasu@openai.com

Abstract

Natural language understanding comprises a wide range of diverse tasks such as textual entailment, question answering, semantic similarity assessment, and document classification. Although large unlabeled text corpora are abundant, labeled data for learning these specific tasks is scarce, making it challenging for discriminatively trained models to perform adequately. We demonstrate that large gains on these tasks can be realized by *generative pre-training* of a language model on a diverse corpus of unlabeled text, followed by *discriminative fine-tuning* on each specific task. In contrast to previous approaches, we make use of task-aware input transformations during fine-tuning to achieve effective transfer while requiring minimal changes to the model architecture. We demonstrate the effectiveness of our approach on a wide range of benchmarks for natural language understanding. Our general task-agnostic model outperforms discriminatively trained models that use architectures specifically crafted for each task, significantly improving upon the state of the art in 9 out of the 12 tasks studied. For instance, we achieve absolute improvements of 8.9% on commonsense reasoning (Stories Cloze Test), 5.7% on question answering (RACE), and 1.5% on textual entailment (MultiNLI).



BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Jacob Devlin **Ming-Wei Chang** **Kenton Lee** **Kristina Toutanova**
Google AI Language
{jacobdevlin,mingweichang,kentonl,kristout}@google.com

Abstract

We introduce a new language representation model called **BERT**, which stands for **B**idirectional **E**ncoder **R**epresentations from **T**ransformers. Unlike recent language representation models (Peters et al., 2018a; Radford et al., 2018), BERT is designed to pre-train deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers. As a result, the pre-trained BERT model can be fine-tuned with just one additional output layer to create state-of-the-art models for a wide range of tasks, such as question answering and language inference, without substantial task-specific architecture modifications.

BERT is conceptually simple and empirically powerful. It obtains new state-of-the-art results on eleven natural language processing tasks, including pushing the GLUE score to 80.5% (7.7% point absolute improvement), MultiNLI accuracy to 86.7% (4.6% absolute improvement), SQuAD v1.1 question answering Test F1 to 93.2 (1.5 point absolute improvement) and SQuAD v2.0 Test F1 to 83.1 (5.1 point absolute improvement).

1 Introduction

Language model pre-training has been shown to be effective for improving many natural language processing tasks (Dai and Le, 2015; Peters et al., 2018a; Radford et al., 2018; Howard and Ruder, 2018). These include sentence-level tasks such as natural language inference (Bowman et al., 2015; Williams et al., 2018) and paraphrasing (Dolan and Brockett, 2005), which aim to predict the relationships between sentences by analyzing them holistically, as well as token-level tasks such as

There are two existing strategies for applying pre-trained language representations to downstream tasks: *feature-based* and *fine-tuning*. The feature-based approach, such as ELMo (Peters et al., 2018a), uses task-specific architectures that include the pre-trained representations as additional features. The fine-tuning approach, such as the Generative Pre-trained Transformer (OpenAI GPT) (Radford et al., 2018), introduces minimal task-specific parameters, and is trained on the downstream tasks by simply fine-tuning *all* pre-trained parameters. The two approaches share the same objective function during pre-training, where they use unidirectional language models to learn general language representations.

We argue that current techniques restrict the power of the pre-trained representations, especially for the fine-tuning approaches. The major limitation is that standard language models are unidirectional, and this limits the choice of architectures that can be used during pre-training. For example, in OpenAI GPT, the authors use a left-to-right architecture, where every token can only attend to previous tokens in the self-attention layers of the Transformer (Vaswani et al., 2017). Such restrictions are sub-optimal for sentence-level tasks, and could be very harmful when applying fine-tuning based approaches to token-level tasks such as question answering, where it is crucial to incorporate context from both directions.

In this paper, we improve the fine-tuning based approaches by proposing BERT: **B**idirectional **E**ncoder **R**epresentations from **T**ransformers. BERT alleviates the previously mentioned unidirectionality constraint by using a "masked language model" (MLM) pre-training objective, in-

BERT: Pre-training of Deep Bidirectional Transformers for ...

by J Devlin · 2018 · Cited by 108443 — We introduce a new **language** representation model

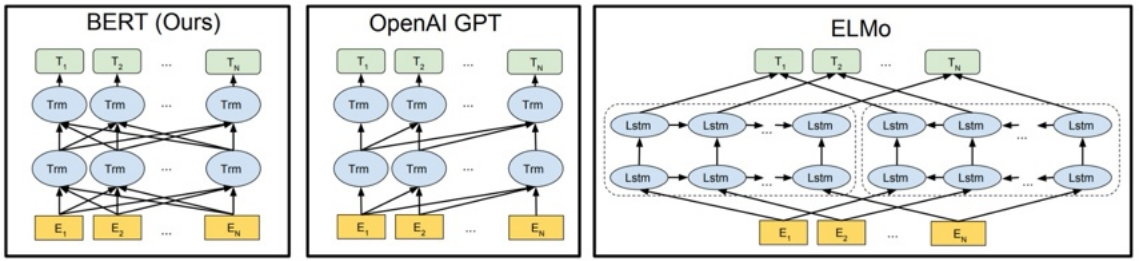
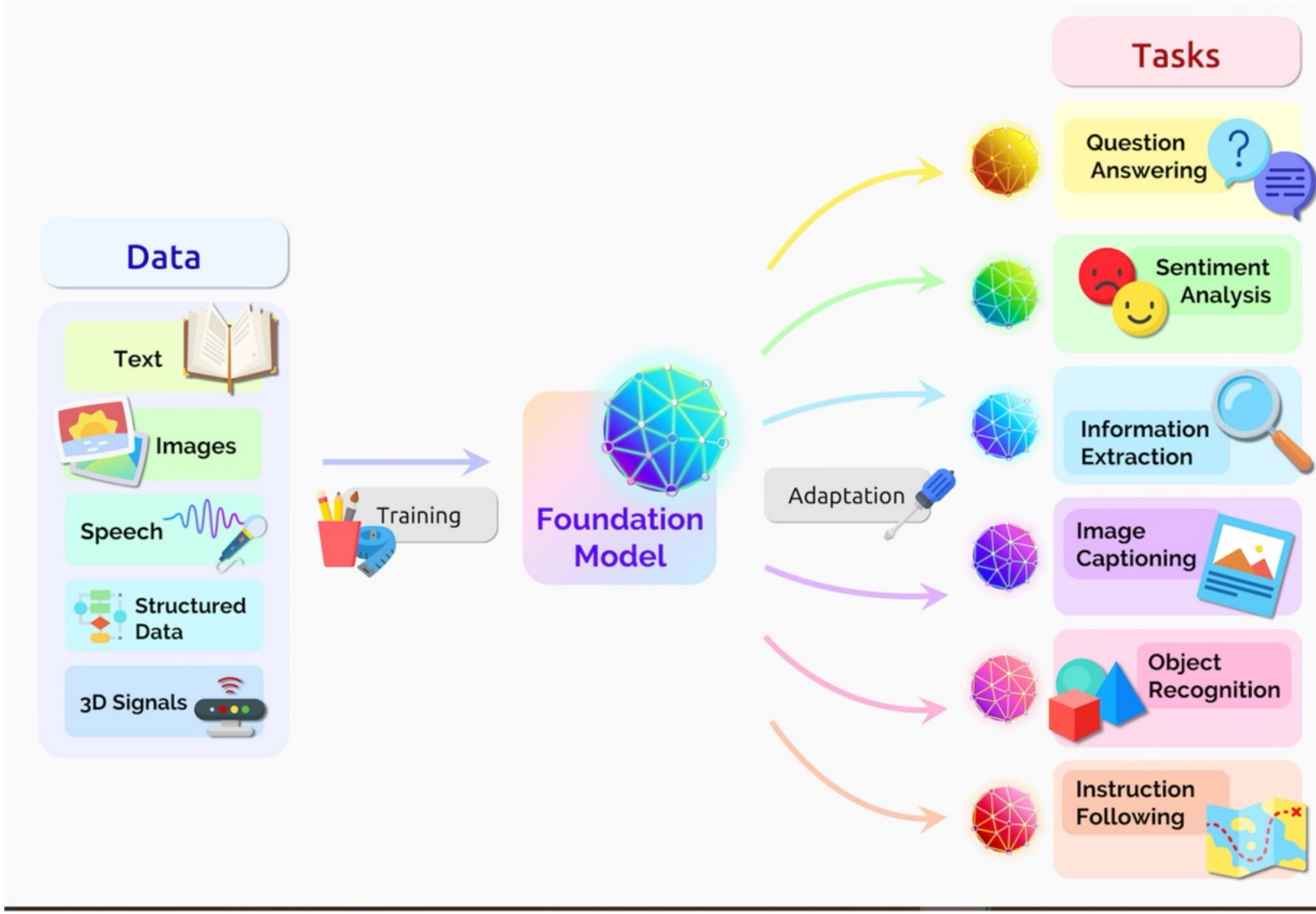


Figure 3: Differences in pre-training model architectures. BERT uses a bidirectional Transformer. OpenAI GPT uses a left-to-right Transformer. ELMo uses the concatenation of independently trained left-to-right and right-to-left LSTMs to generate features for downstream tasks. Among the three, only BERT representations are jointly conditioned on both left and right context in all layers. In addition to the architecture differences, BERT and OpenAI GPT are fine-tuning approaches, while ELMo is a feature-based approach.

3. Using BERT for Different NLP Tasks

- **Text Classification**: BERT can be used for tasks like sentiment analysis, spam detection, or topic classification. After fine-tuning, the `[CLS]` token's output (the special token at the start of the sequence) is passed through a classifier to predict the label of the text.
- **Named Entity Recognition (NER)**: BERT can be fine-tuned to recognize and classify named entities (e.g., names, organizations, dates) in text. Each token's output from BERT is passed through a classification layer to predict the entity label.
- **Question Answering**: For tasks like SQuAD (Stanford Question Answering Dataset), BERT can be used to find the answer to a question within a given passage. The model is fine-tuned to predict the start and end positions of the answer span in the passage.
- **Sentence Pair Classification**: Tasks like entailment (e.g., recognizing textual entailment) or semantic textual similarity can be handled by BERT. Two sentences are input, and BERT is fine-tuned to classify their relationship (e.g., entailment, contradiction, neutral).
- **Text Generation**: While BERT itself is not primarily a text generation model (since it's not designed to generate text sequentially), it can be adapted for certain text completion tasks or used in conjunction with other models for generation.


Data
Text
Images
Speech
Structured Data
3D Signals
Training
Foundation Model
Adaptation
Tasks
Question Answering
Sentiment Analysis
Information Extraction
Image Captioning
Object Recognition
Instruction Following

Discussion

Transformers and related architectures are transforming language processing and related tasks: attention is all you need

- Attention finds for a given term which other terms are important
- Transformers are great for language processing by computing progressively more abstract features and concepts in a very high dimensional space
- Transformer are the core of agentic operations: LLM respond to prompts, give responses that the “agentic harness” then parses to perform actions

Questions:

- DNA sequences share some properties with natural language
- Natural language has a lot of semantics, and is constrained by human brain
- If we apply to bioinformatics, do the same semantics and constraints apply?
- Some parts yes (Enformer is working), some parts need more research!

Deep Learning in Genomics

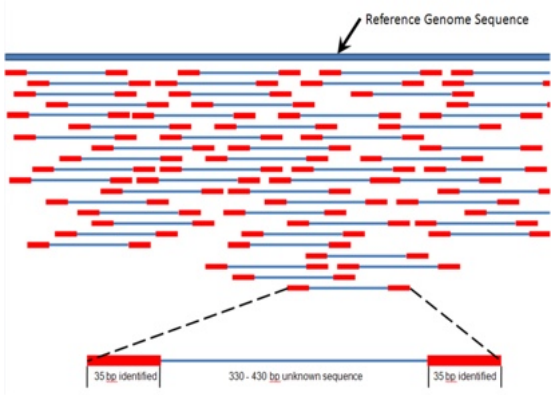
Imagine you are a doctor treating a patient with some condition



You sequence their DNA…



…align it to the reference genome…



…and find all of their variants

```
##fileformat=VCFv4.1
##fileDate=20140930
##source=23andme2vcf.pl https://github.com/arrogantrobot/23
##reference=file://23andme_v3_hg19_ref.txt.gz
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype"
#CHROM  POS ID  REF ALT QUAL    FILTER  INFO    FORMAT  GEN
chr1    82154   rs4477212   a   .   .   .   .   GT  0/0
chr1    752566  rs3094315   g   A   .   .   .   GT  1/1
chr1    752721  rs3131972   A   G   .   .   .   GT  1/1
chr1    798959  rs11240777  g   .   .   .   .   GT  0/0
chr1    800007  rs6681049   T   C   .   .   .   GT  1/1
chr1    838555  rs4970383   c   .   .   .   .   GT  0/0
chr1    846808  rs4475691   C   .   .   .   .   GT  0/0
chr1    854250  rs7537756   A   .   .   .   .   GT  0/0
chr1    861808  rs13302982  A   G   .   .   .   GT  1/1
chr1    873558  rs1110052   G   T   .   .   .   GT  1/1
chr1    882033  rs2272756   G   A   .   .   .   GT  0/1
chr1    888659  rs3748597   T   C   .   .   .   GT  1/1
chr1    891945  rs13303106  A   G   .   .   .   GT  0/1
```

How do you determine which variant(s) are responsible for their condition?

What does a DNA variant do?

Method	Drawback(s)
Query it in a database	The variant might not exist in a database The variant might act differently under different conditions
Assume loss-of-function of nearby gene	Not always an accurate assumption Function of nearby gene may not itself be understood
Genome-Wide Association Study (GWAS)	Not applicable for variants that only exist in a single person Underpowered for rare variants that only occur in a few people Relies on undersized datasets containing a small number of SNPs Requires rigorous statistical testing Can only model one variant at a time Assumes additive linear effects of variants Difficult to adapt to a polygenic/omnigenic model of traits In practice, only explains a fraction of heritability

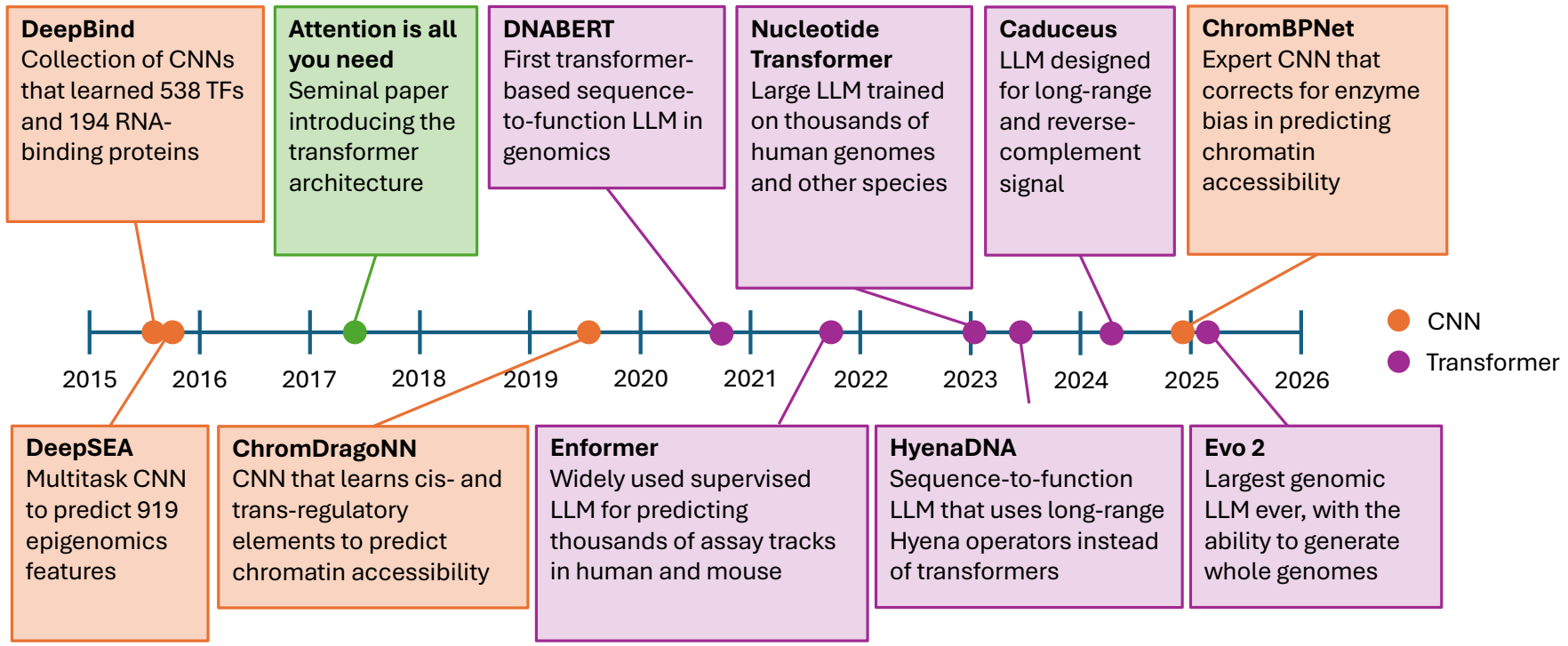
Variant effect prediction

- What **effect** does **variant** X have on us? What **trait** does it create?
- Arguably the most important task in bioinformatics

- Historically studied using Genome-Wide Association Studies (**GWAS**) in connection with Quantitative Trait Loci (**QTL**) analysis
  - This approach has many drawbacks

- Could be studied using deep learning

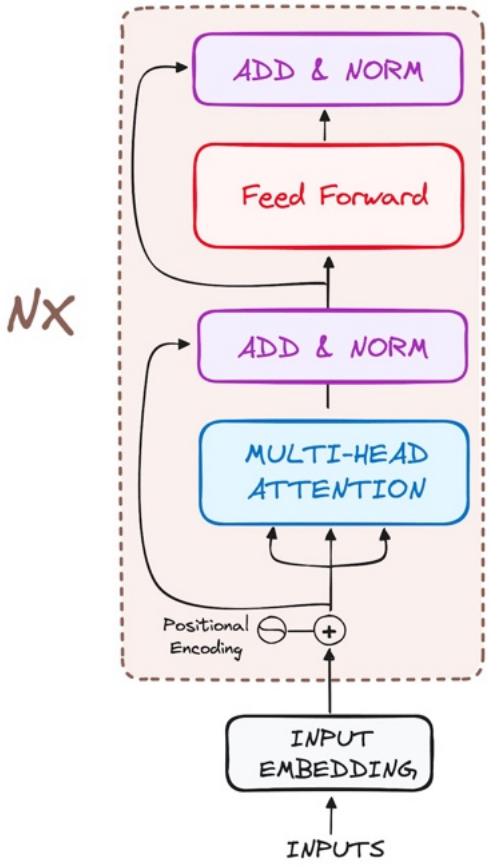
Deep learning is now widespread in genomics



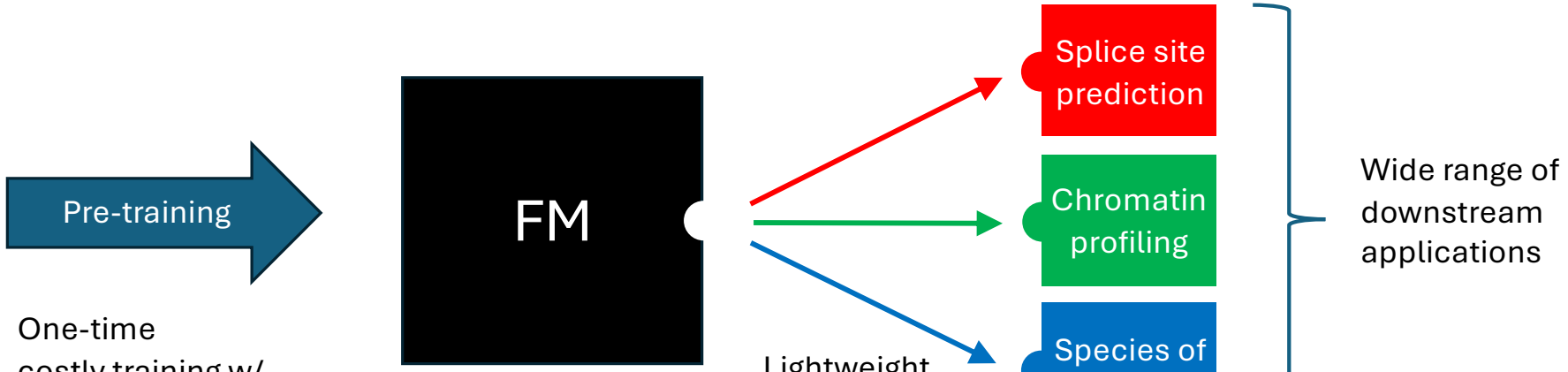
https://www.datacamp.com/tutorial/how-transformers-work

Why use LLMs in genomics?

- LLMs employ the most advanced **architecture** ever made for AI
  - Core advancement is **Attention**
    - First truly effective way to process **sequential** data like **text**
  - Attention allows models to get really **big**, and bigger is better
- Operate at **lower cost** than humans
- Can conduct massively **parallel** experiments with **little human effort**
- Can model **rare** or **novel sequences**



Foundation model (FM)



One-time costly training w/ **billions** of training samples

e.g. train on the **entire human genome**

Versatile, **reusable** Foundation Model

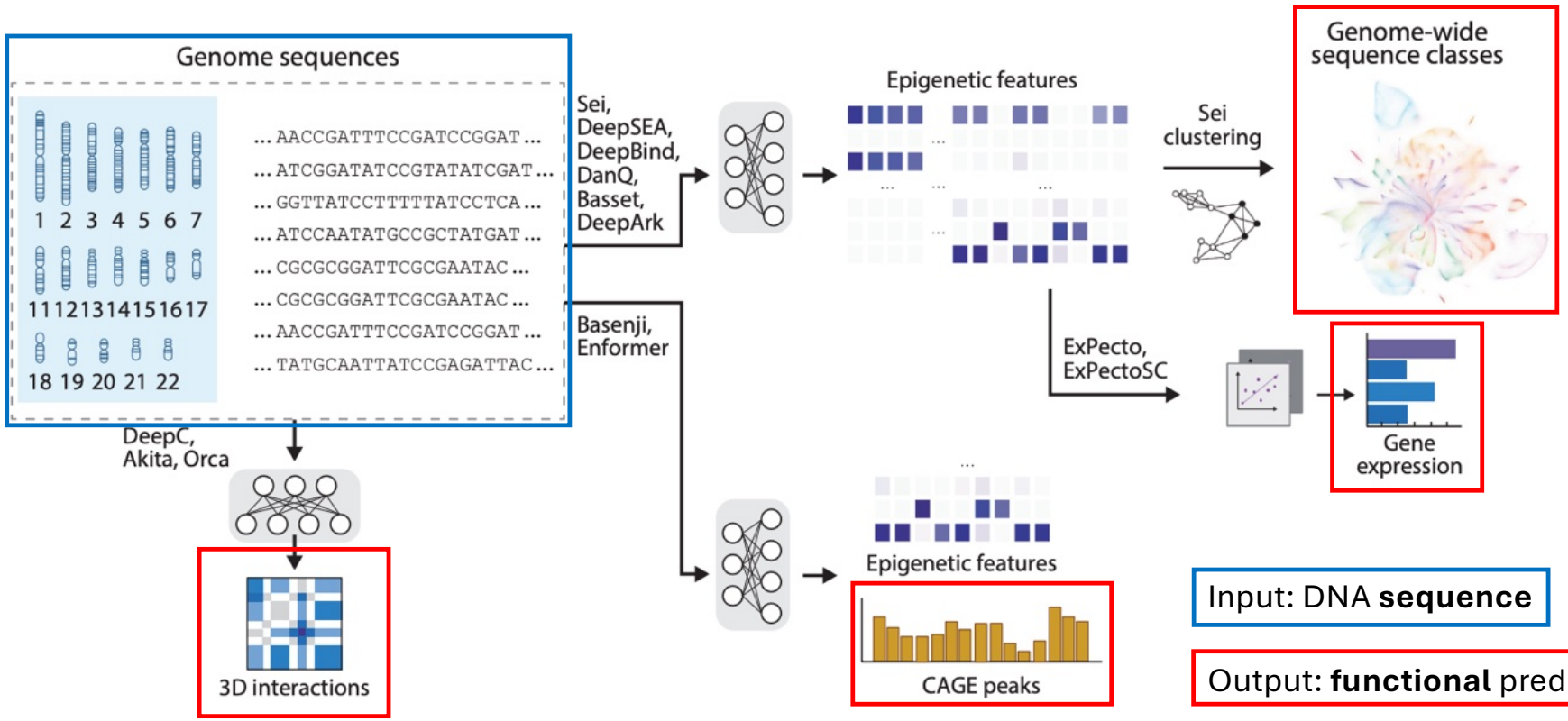
Pre-train once, then fine-tune many times

Lightweight finetuning w/ only **thousands** of samples

e.g. train on a **small number** of **labelled** splice sites

Sokolova et al. 2024.

Sequence-to-function models



Enformer

Article | Open access | Published: 04 October 2021

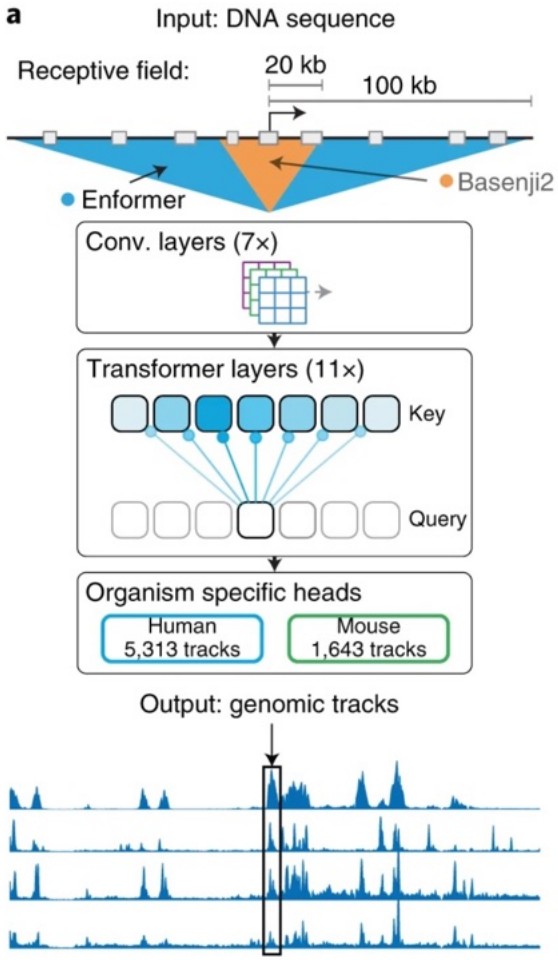
Effective gene expression prediction from sequence by integrating long-range interactions

Žiga Avsec, Vikram Agarwal, Daniel Visentin, Joseph R. Ledsam, Agnieszka Grabska-Barwinska, Kyle R. Taylor, Yannis Assael, John Jumper, Pushmeet Kohli & David R. Kelley

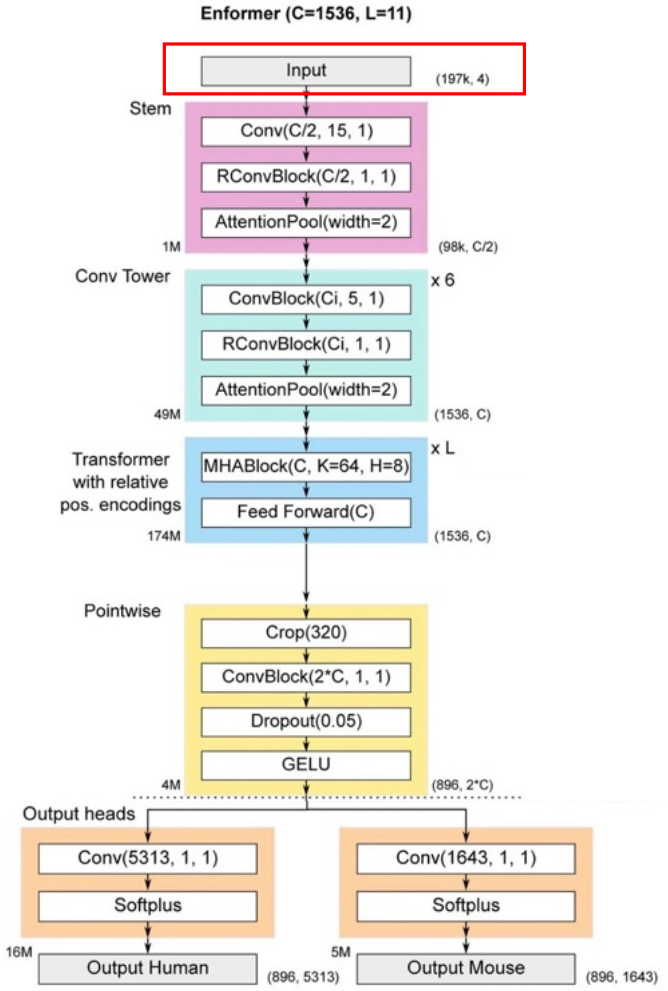
Nature Methods **18**, 1196–1203 (2021) | Cite this article

306k Accesses | **1001** Citations | **380** Altmetric | Metrics

- The most widely used genomic Large Language Model (LLM)
- Developed in the first wave of genomic LLMs
- Has since defined the field



- Trained on **197kb** DNA sequences from the human and mouse **reference** genomes
- Each sequence has thousands of corresponding labels; **supervised** model
  - **ATAC-seq**
    - Is this sequence available to be transcribed?
  - **DNase-seq**
    - Same as ATAC-seq, but different measurement
  - **ChIP-seq**
    - Are proteins binding to this sequence?
  - **CAGE**
    - How much of this sequence is transcribed into RNA?
- Model predicts **all labels** at **each position**

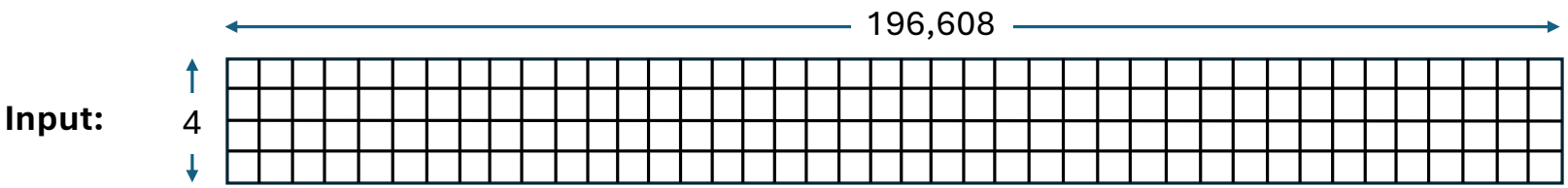


Input is 197k one-hot encoded DNA bases

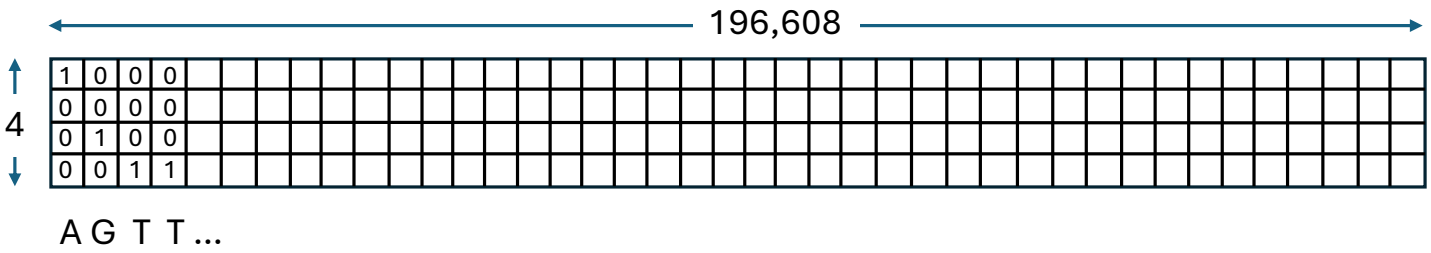
GATTACA

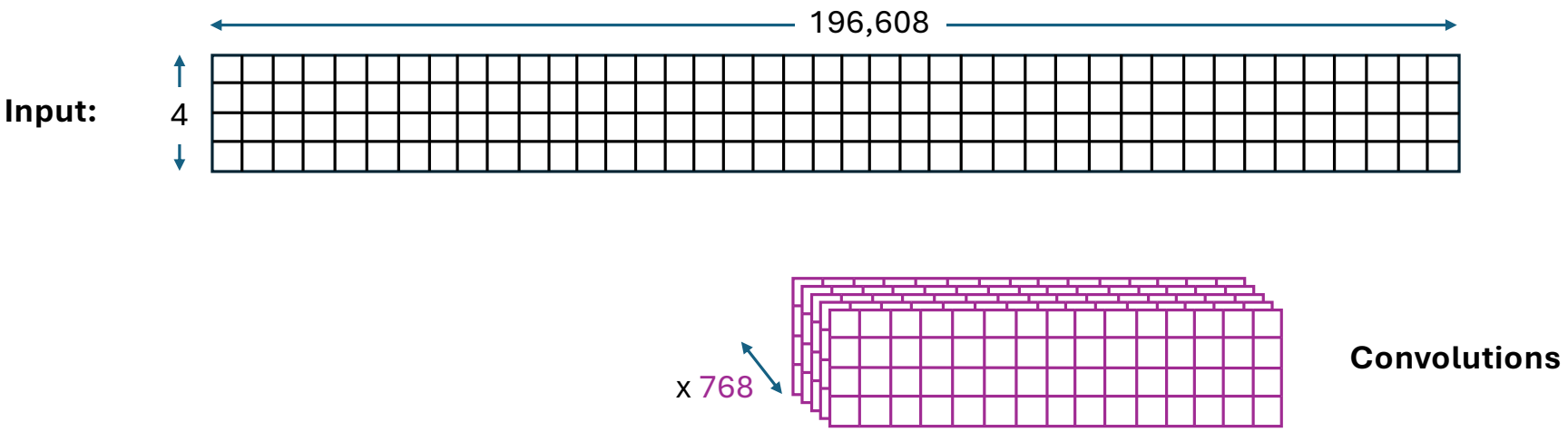
↓

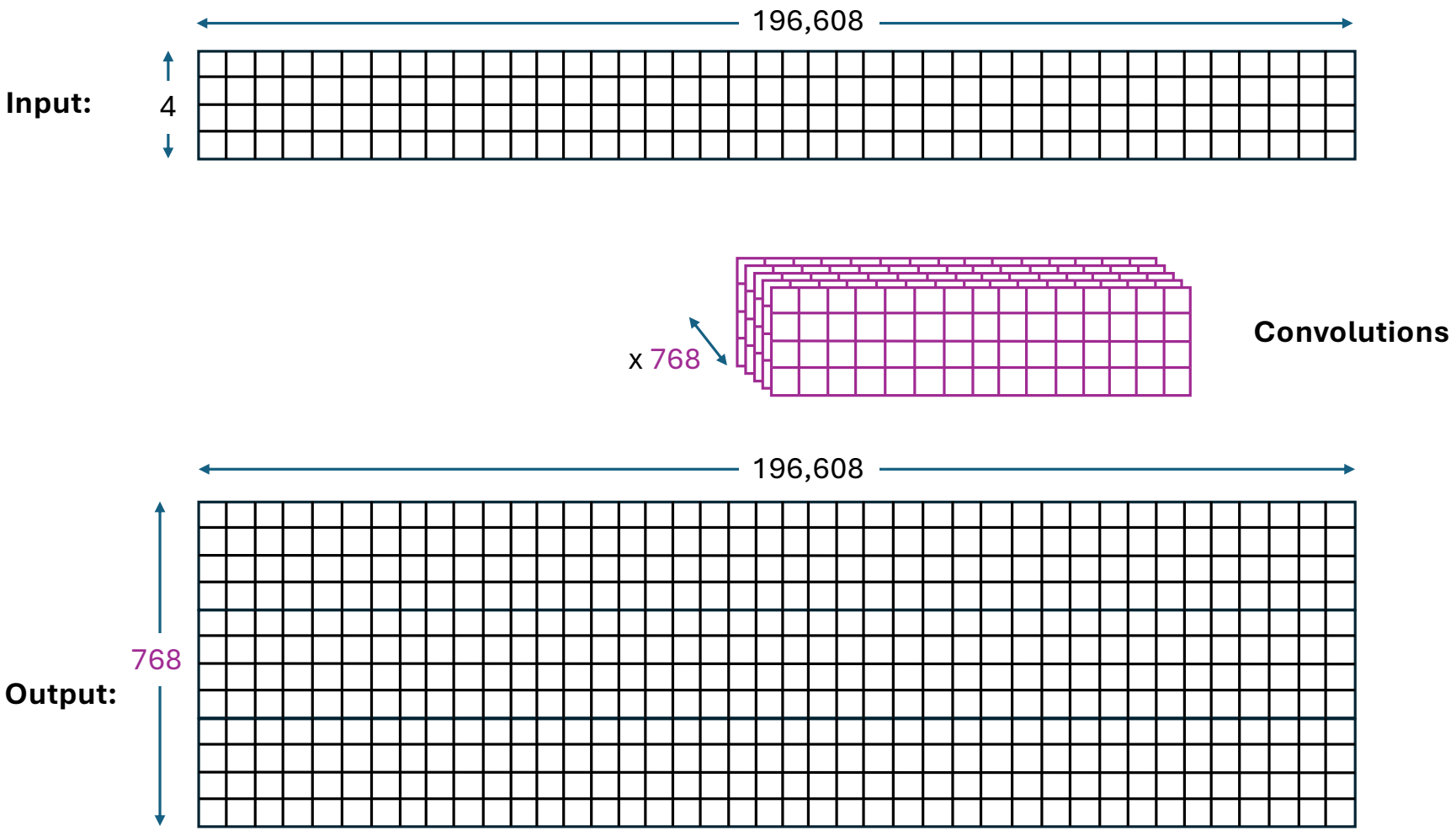
[0 0 1 0], [1 0 0 0], [0 0 0 1], [0 0 0 1], [1 0 0 0], [0 1 0 0], [1 0 0 0]

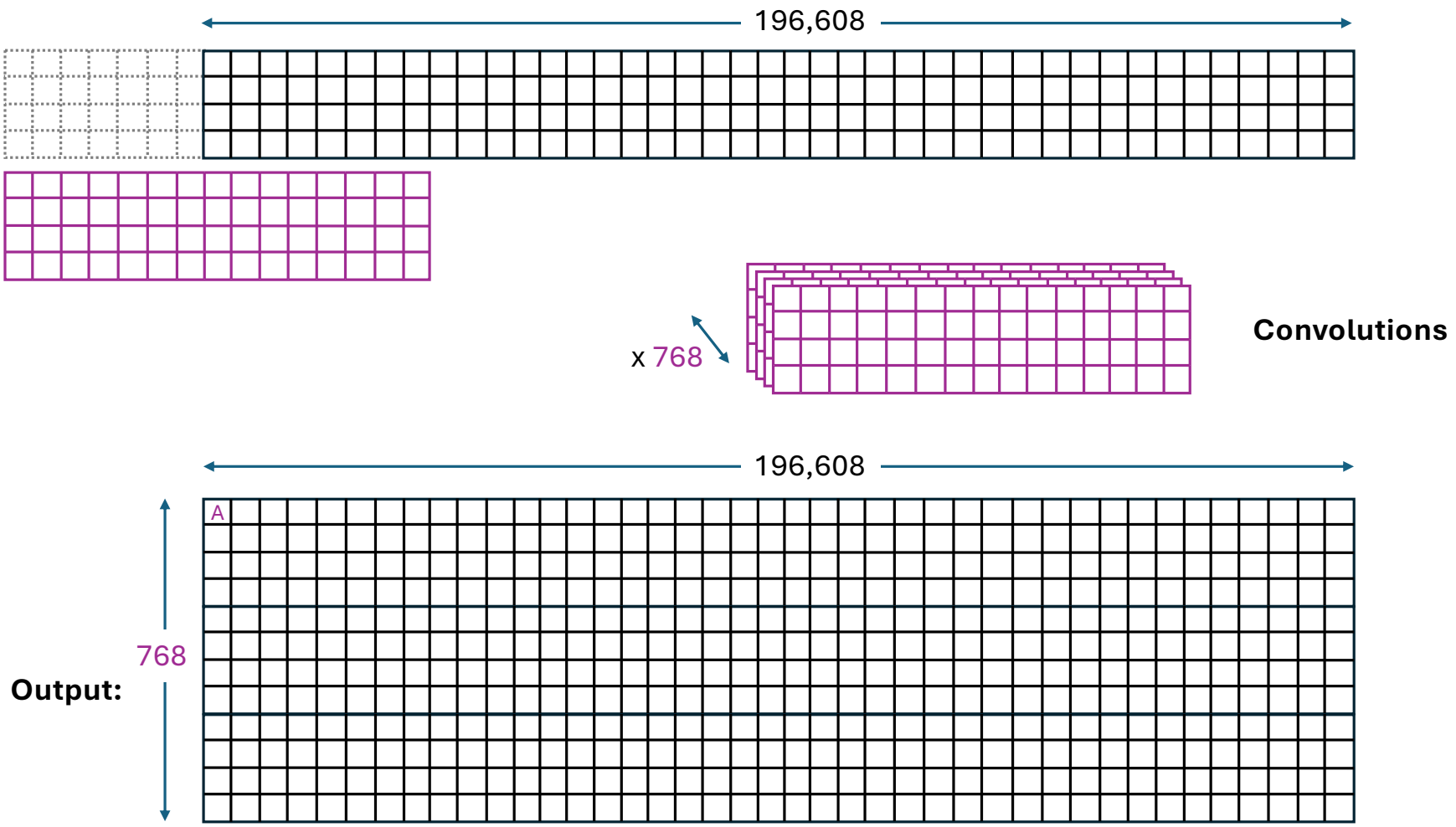

196,608
Input:
4

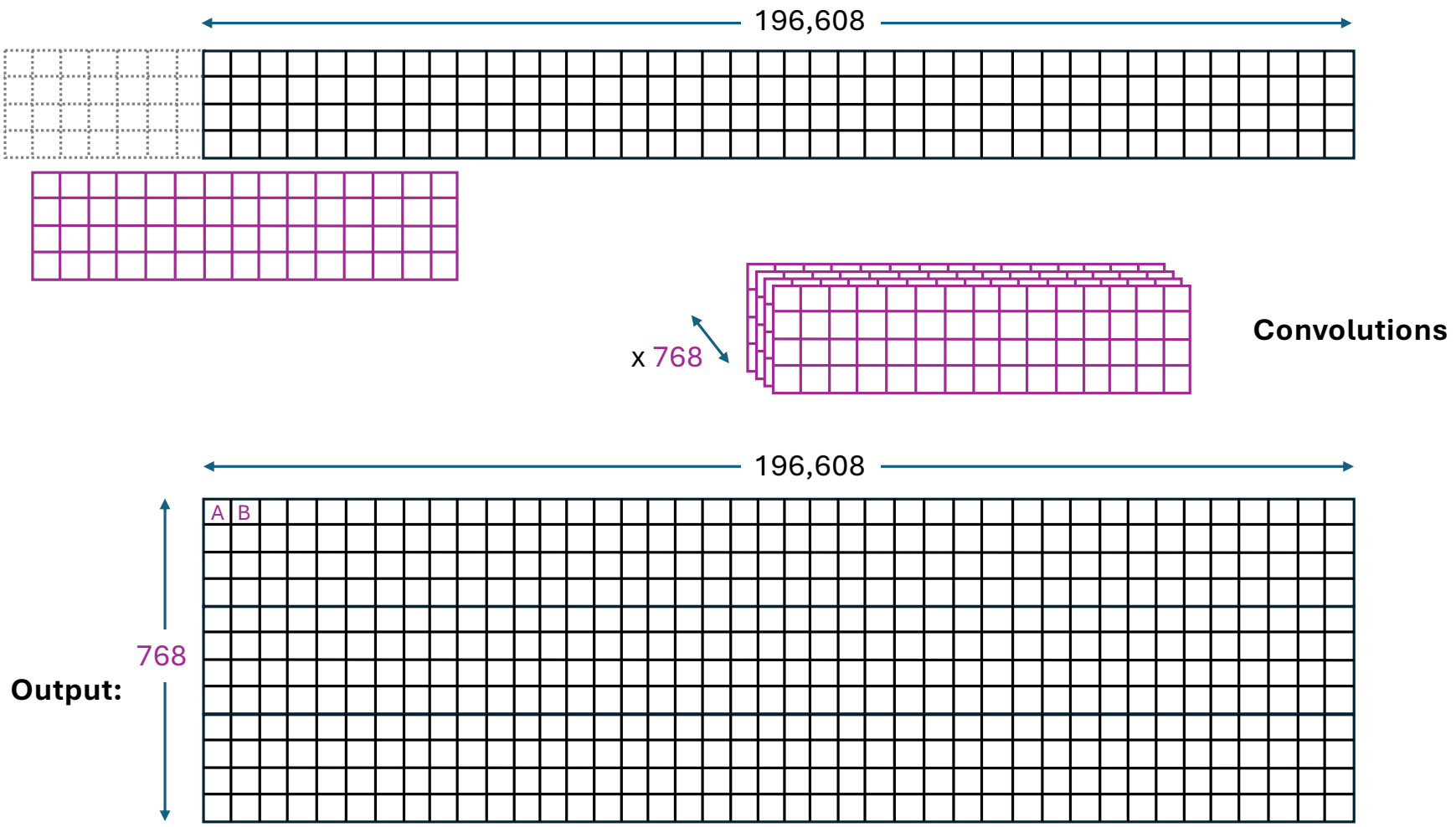
Input:

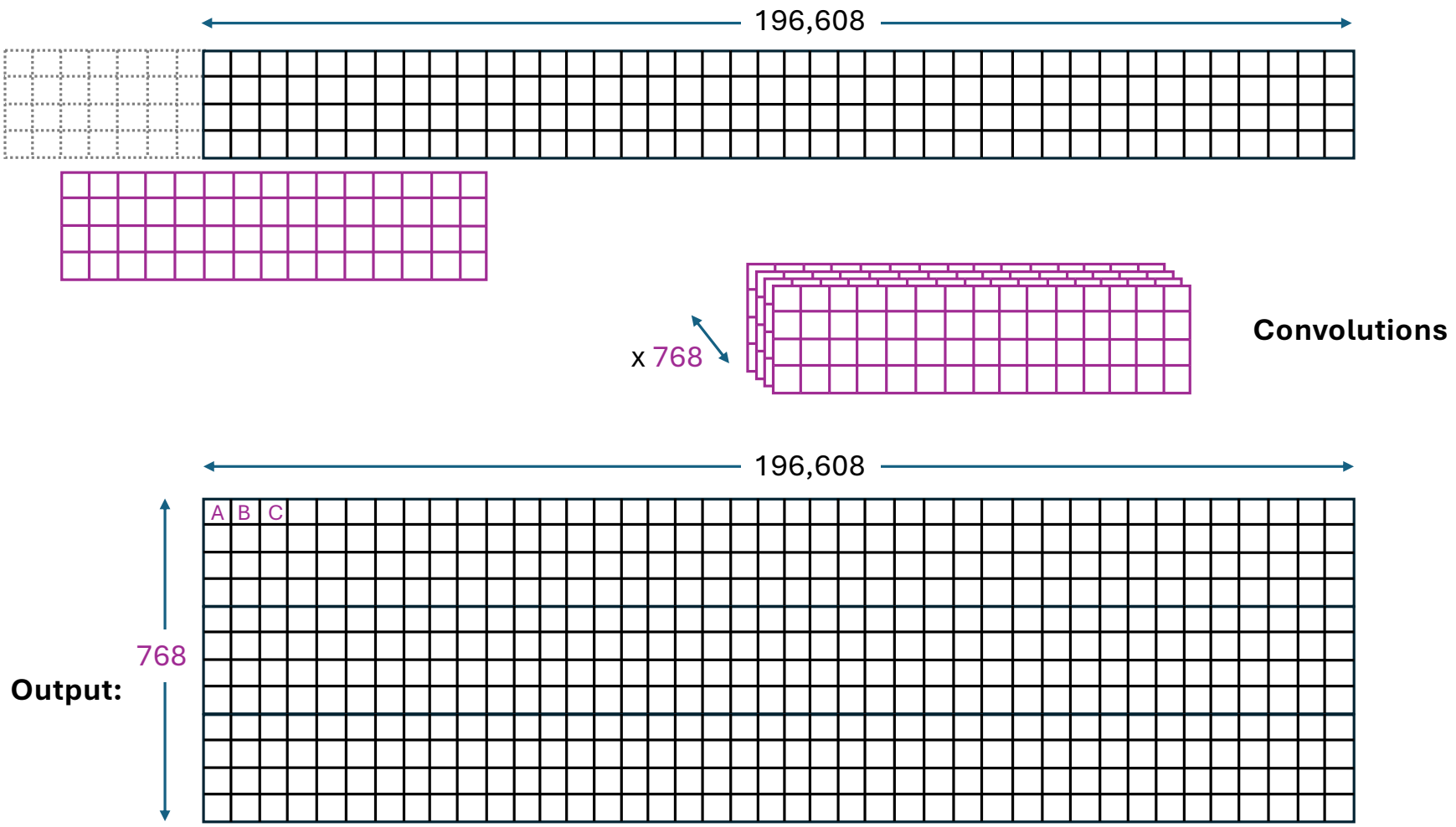


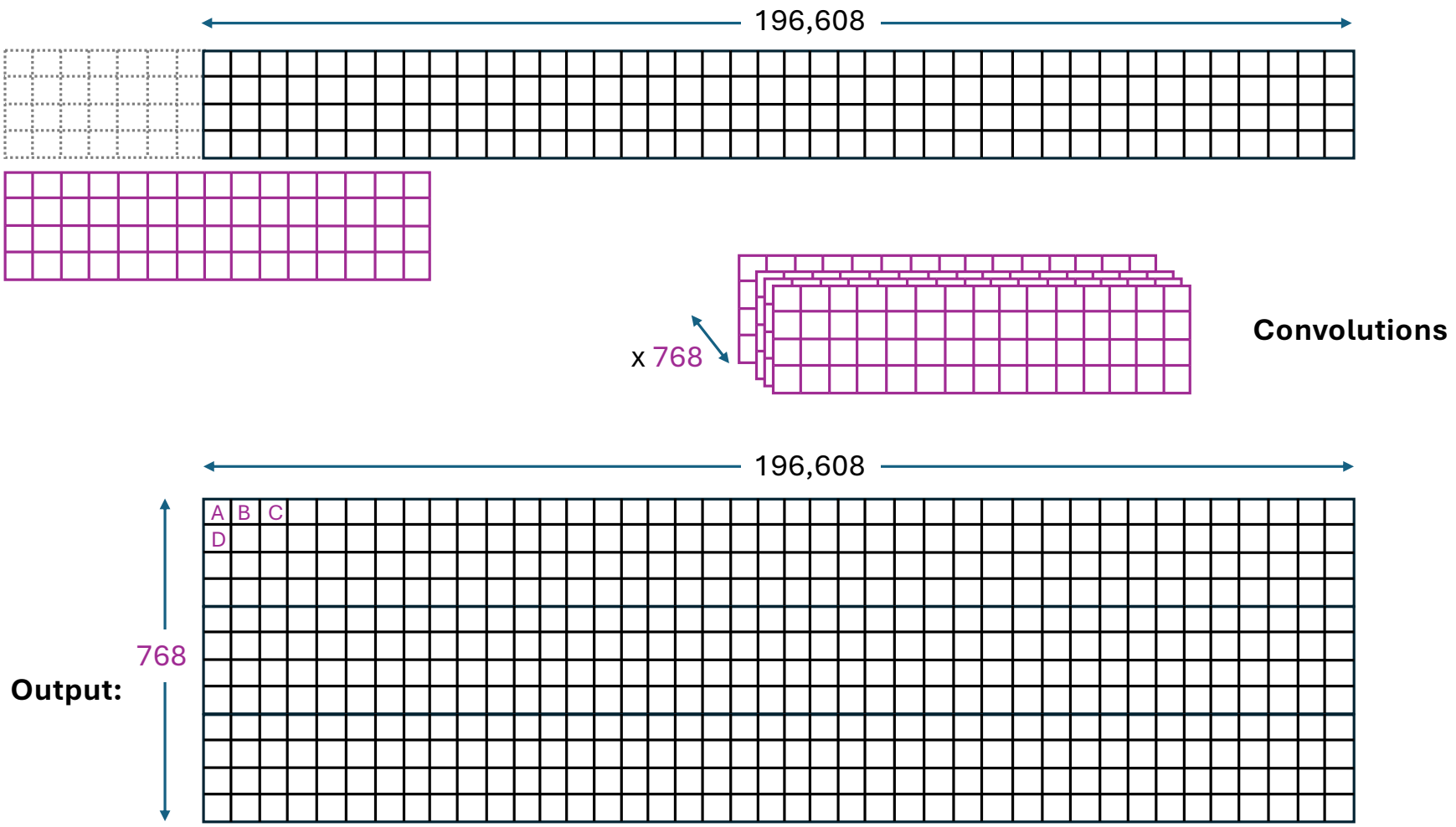

196,608
Input:
4
x 768
Convolutions

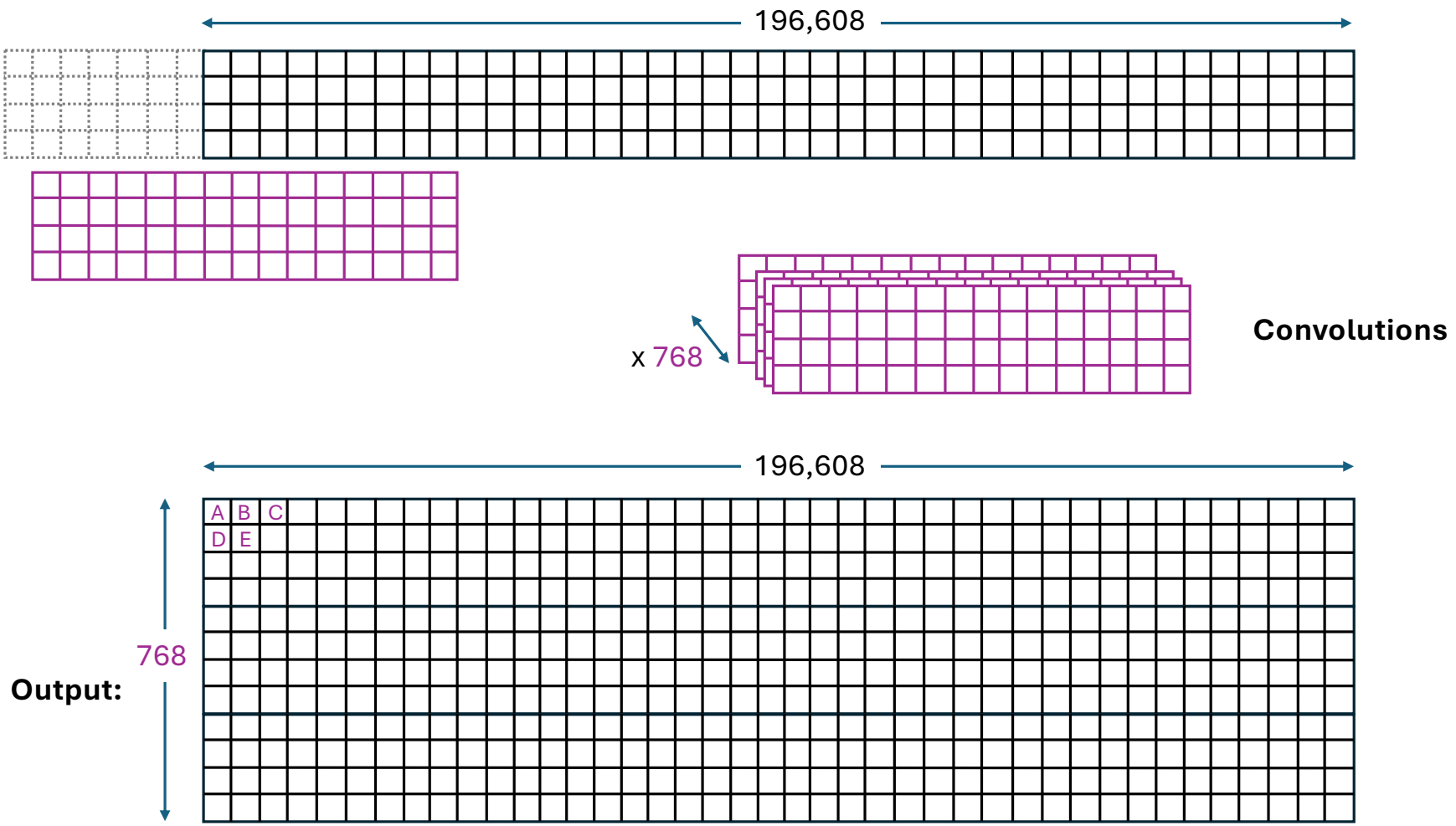

196,608
Input:
4
x 768
Convolutions
196,608
768
Output:

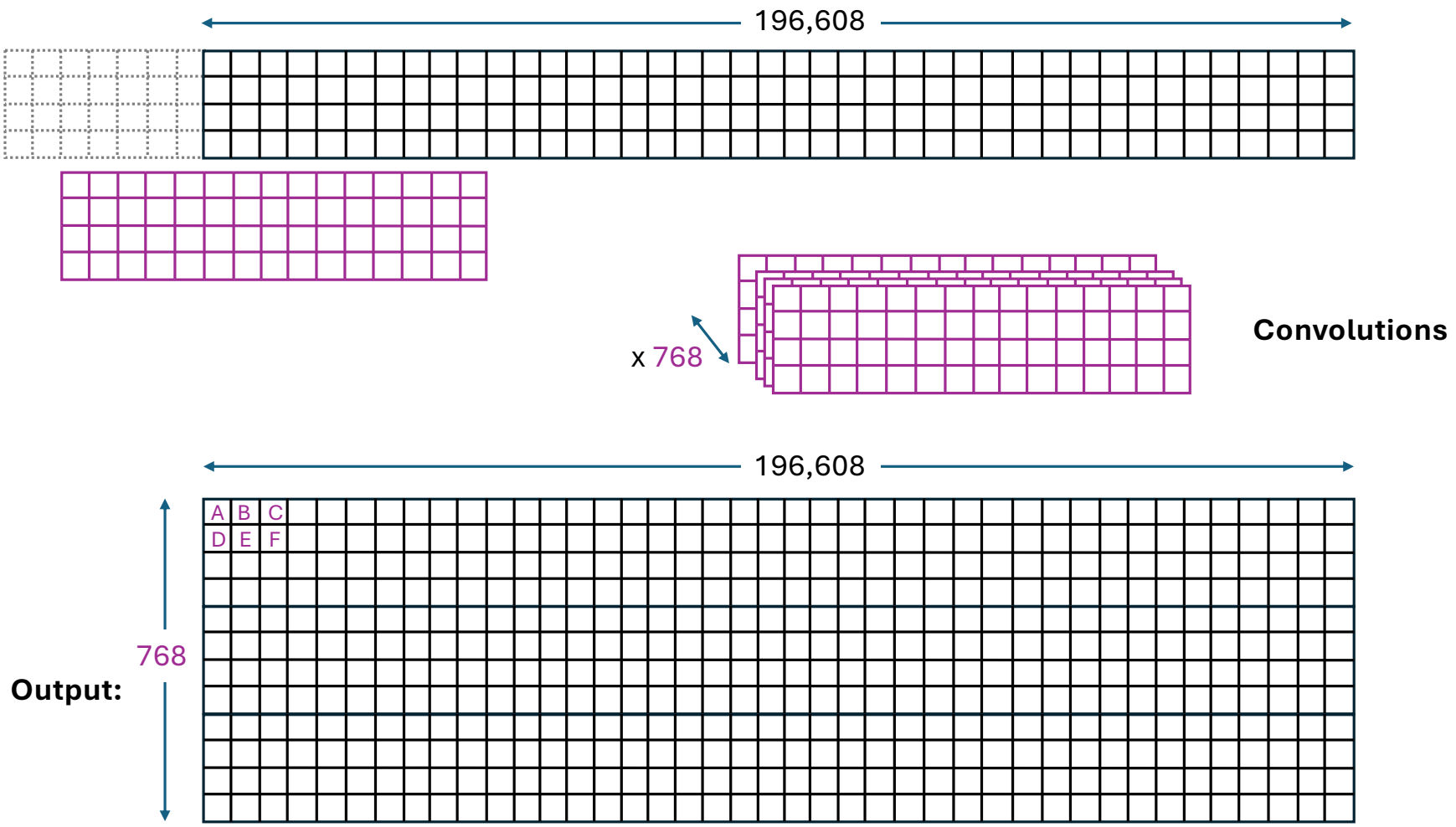

196,608
x 768
Convolutions
196,608
A
768
Output:

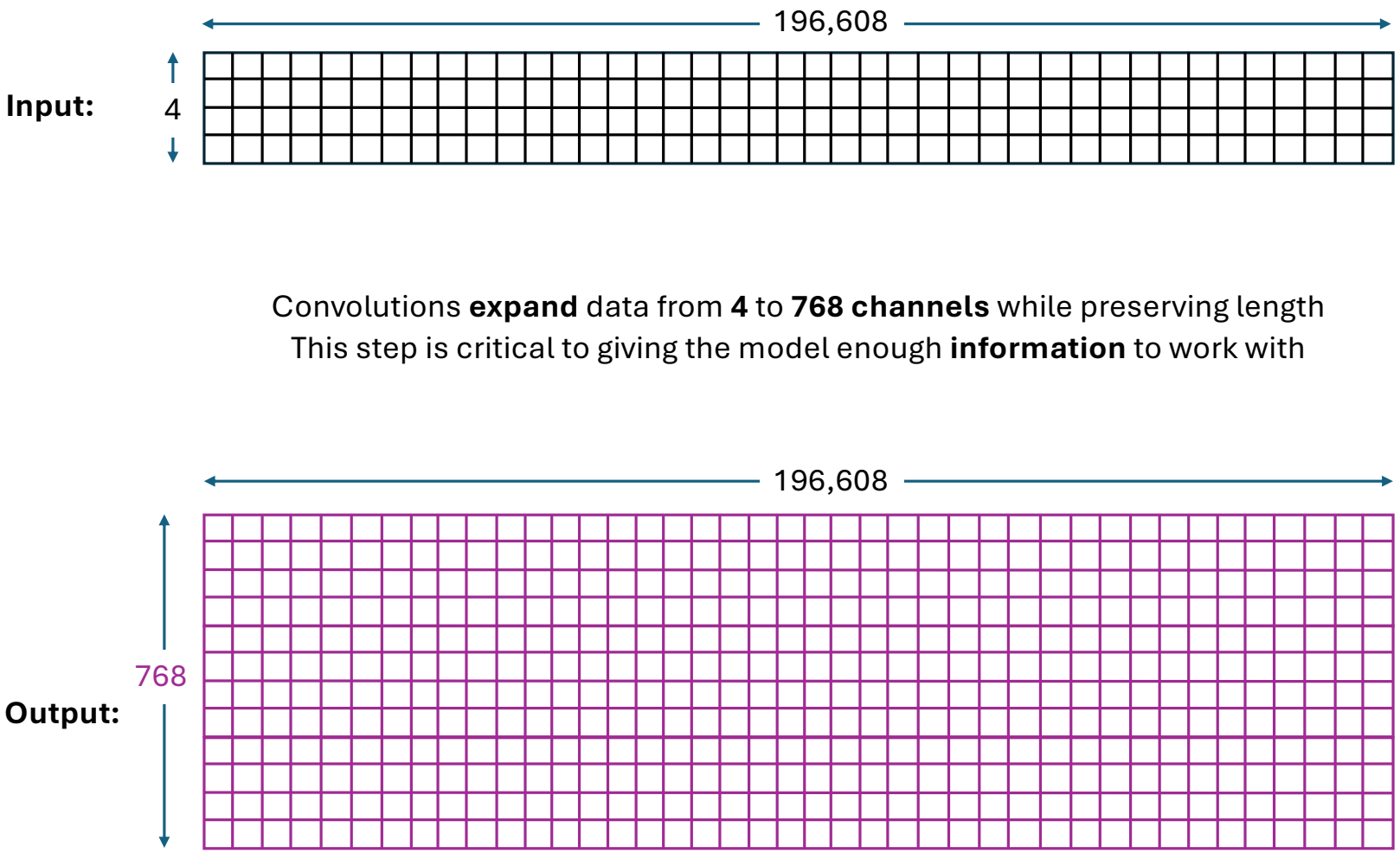

196,608
x 768
Convolutions
196,608
A
B
768
Output:

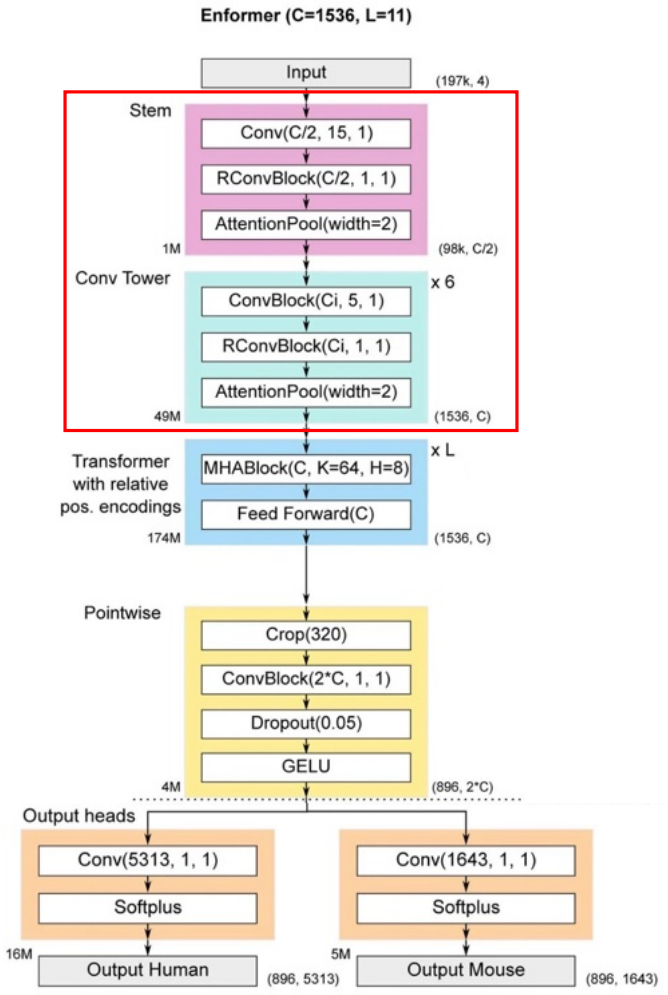

196,608
x 768
Convolutions
196,608
A
B
C
768
Output:


196,608
x 768
Convolutions
196,608
A
B
C
D
768
Output:

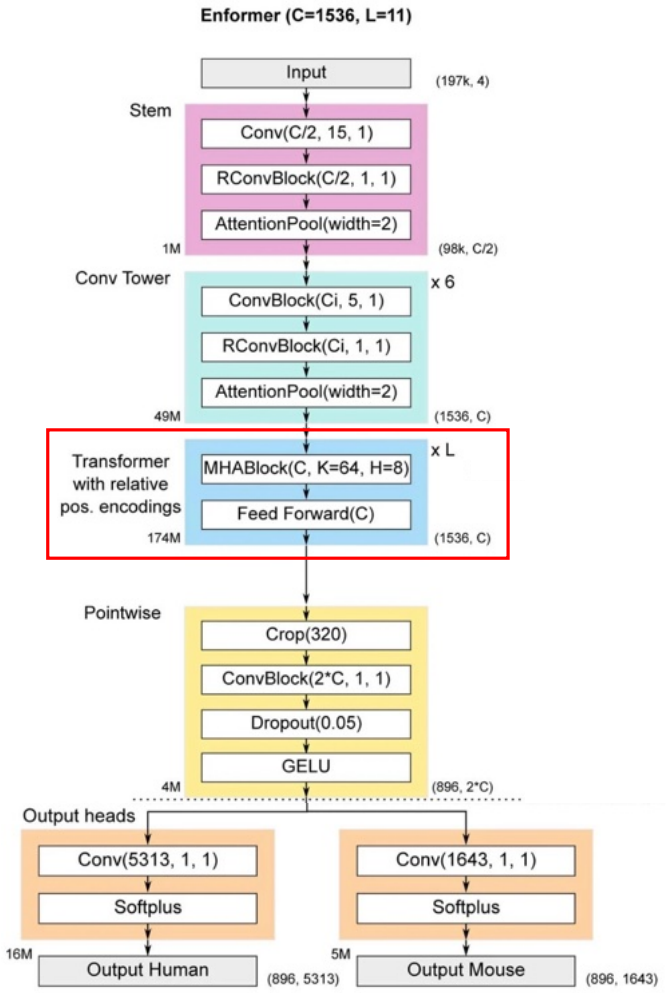

196,608
x 768
Convolutions
196,608
A B C
D E
768
Output:


196,608
x 768
Convolutions
196,608
A B C
D E F
768
Output:

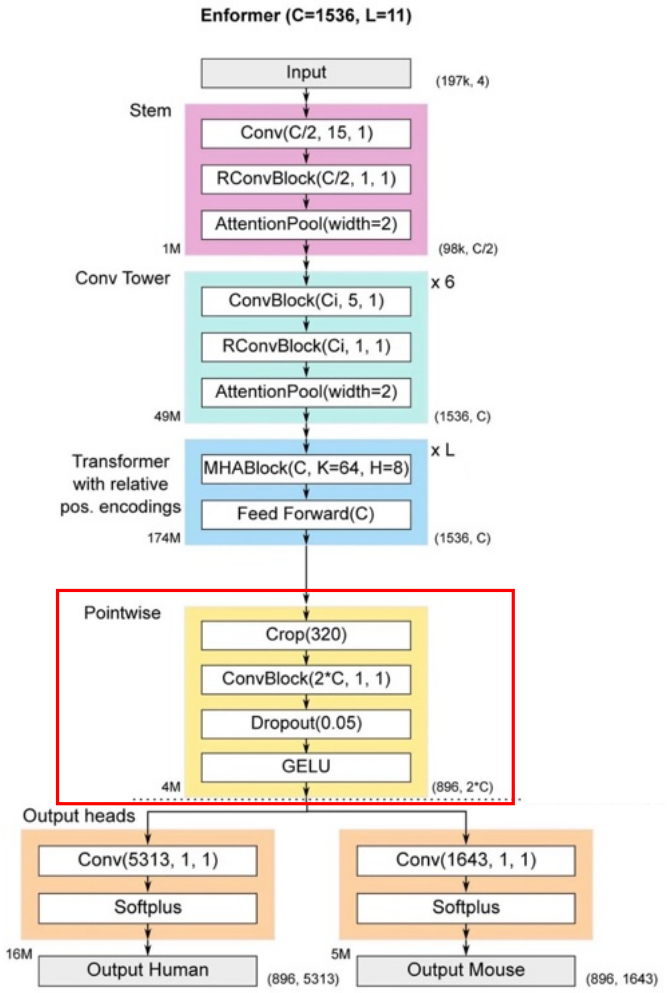

196,608
Input:
4
Convolutions **expand** data from **4** to **768 channels** while preserving length
This step is critical to giving the model enough **information** to work with
196,608
768
Output:



Increase the **dimensionality** of the data (4 -> 1536 channels)
Decrease the **number** of data points (197k ->1536)

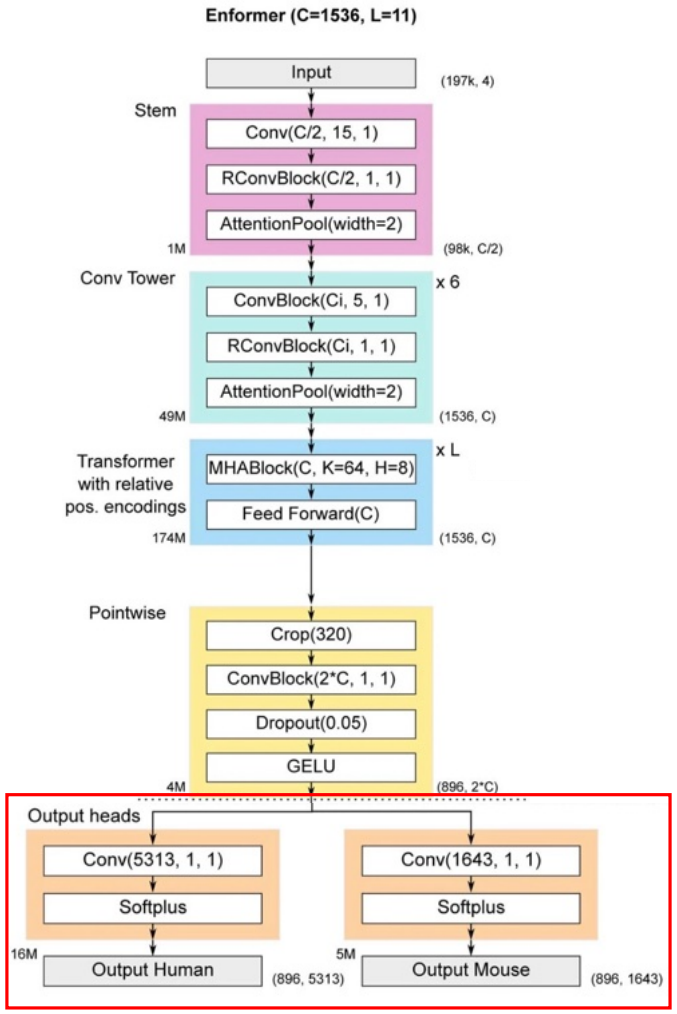


Add **context** to data, like combining word vectors into sentence vectors
The power of transformers



Fully connected network

Learn **patterns** in data

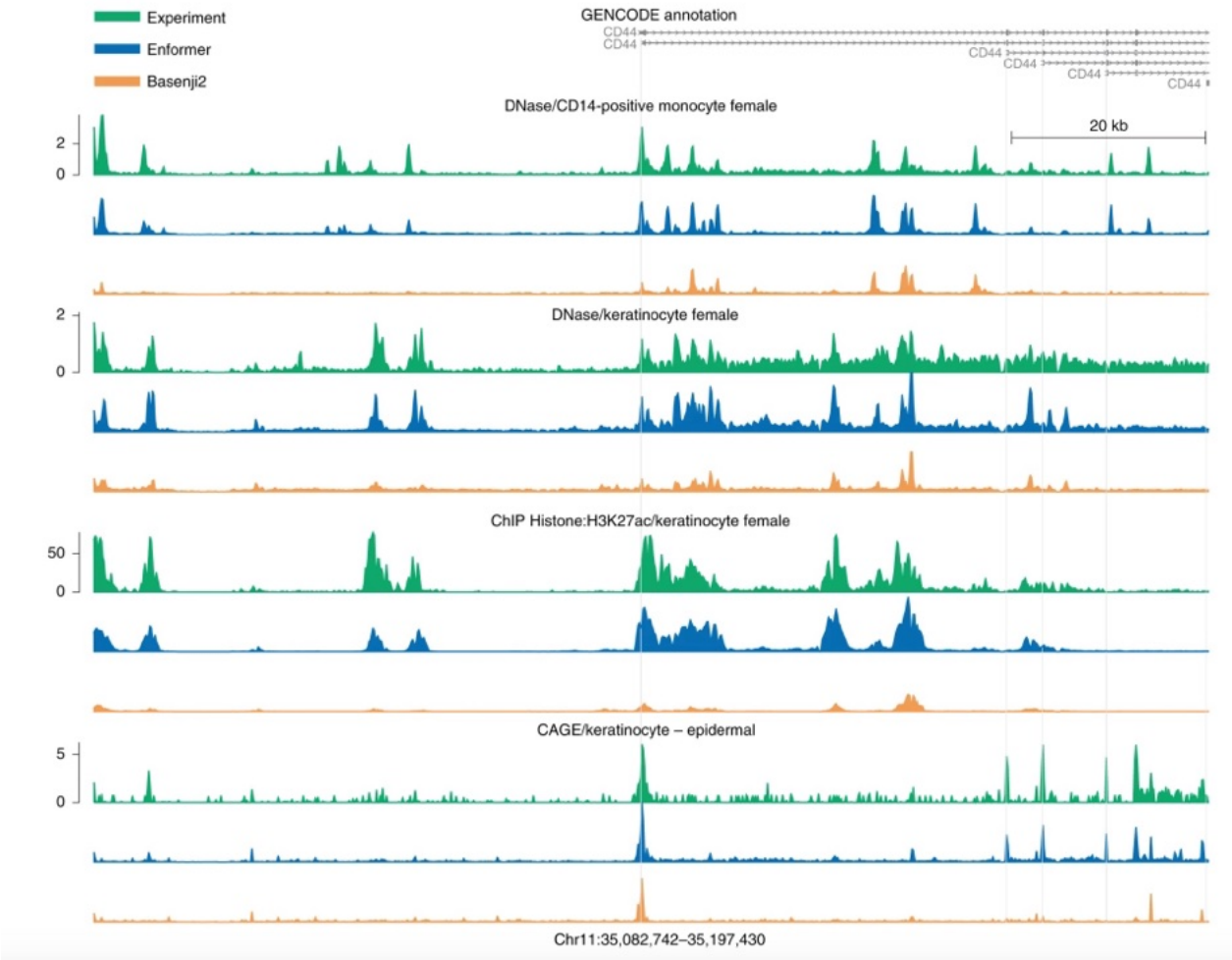


Emit **predictions** across all labels

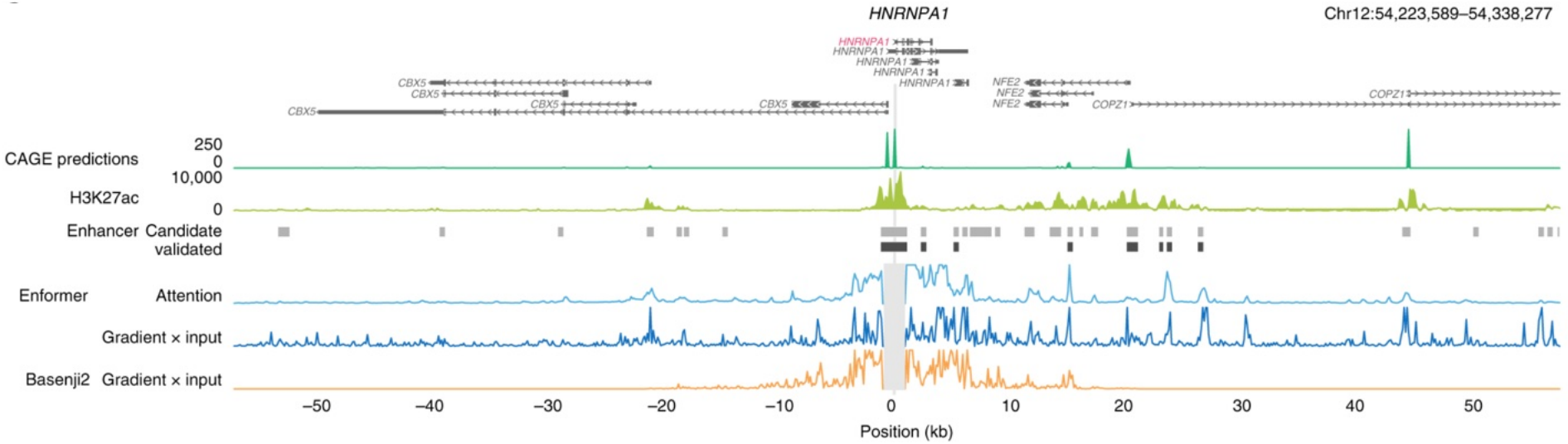
What does Enformer do?

- Green: test sample **experimental** data
- Blue: Enformer's **predictions**

- Training enables it to **generalize** to unseen samples

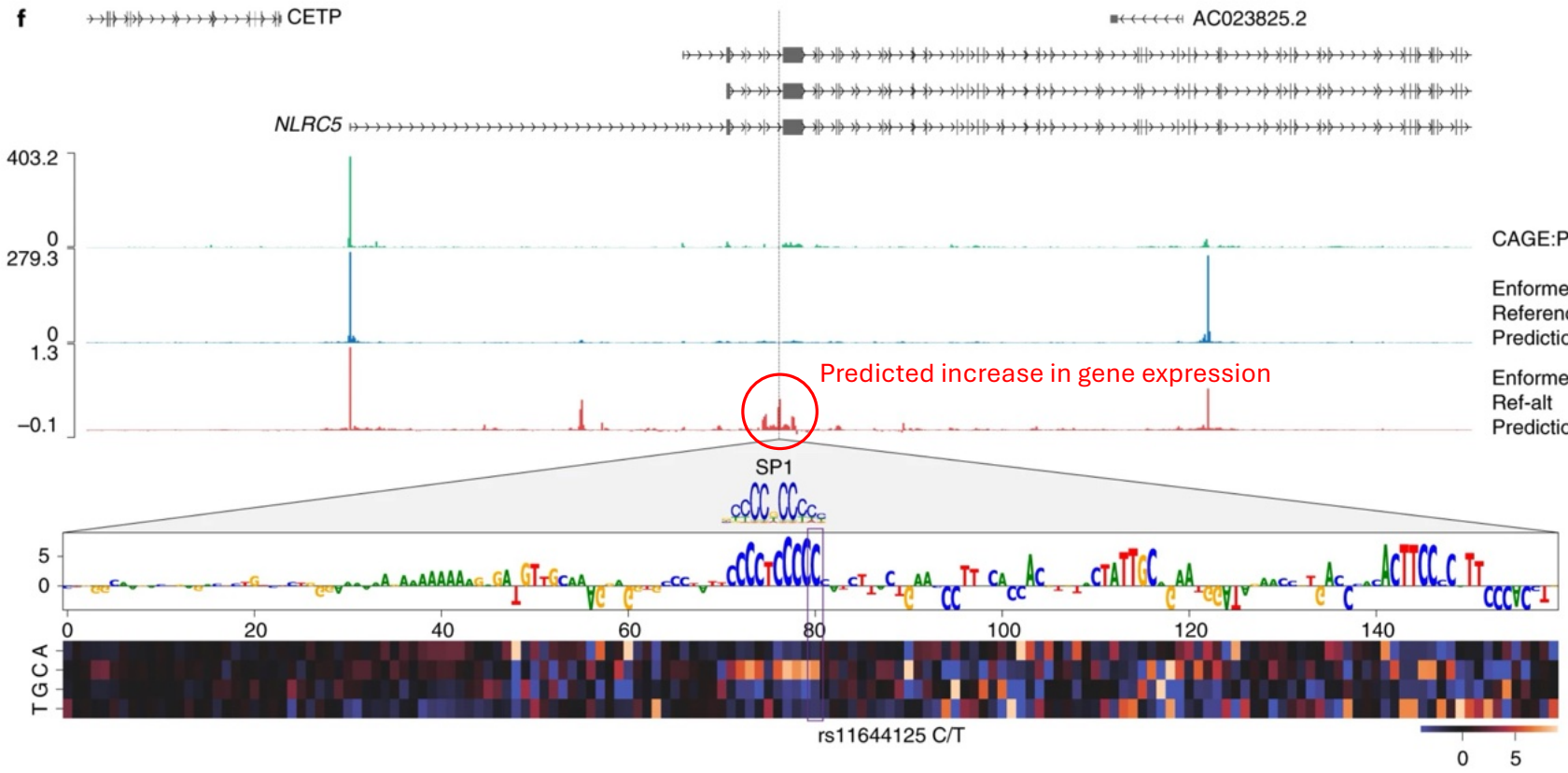


Interpreting Enformer's predictions



- Dark green: Enformer's CAGE **prediction** of transcript abundance for *HNRNPA1*
- Light green and gray: **known** enhancer positions for *HNRNPA1*
- Blues: **attention** and **gradient** values at each position of the input
- Locations of high attention and gradients **correspond** with known enhancers; looking in the **right places**

Understanding variants with Enformer



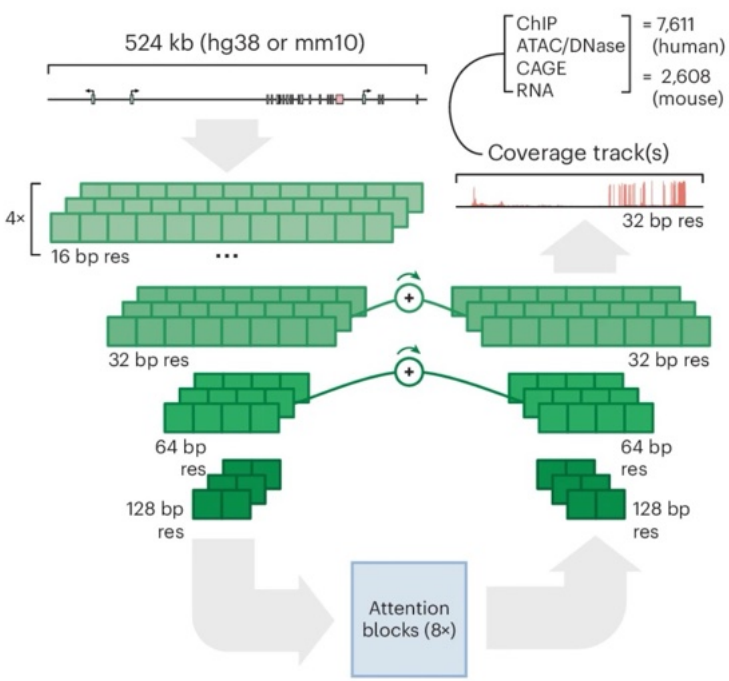
Mutation **disrupts known** SP1 transcription factor binding **motif**

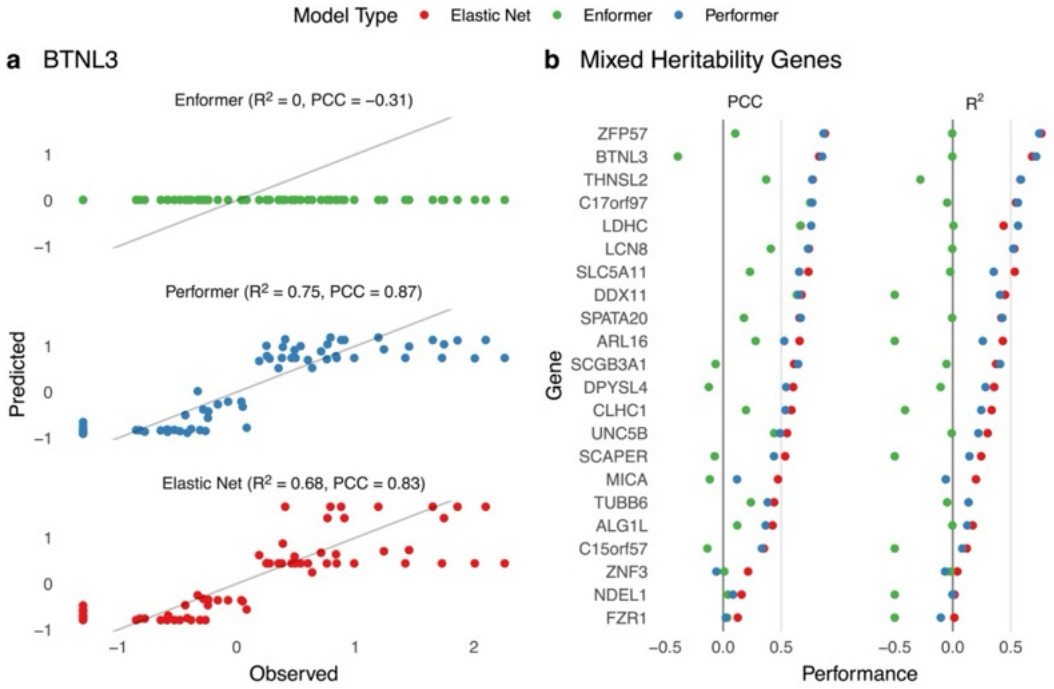
Linder et al. 2025.
Drusinsky et al. 2024.

Enformer's influence

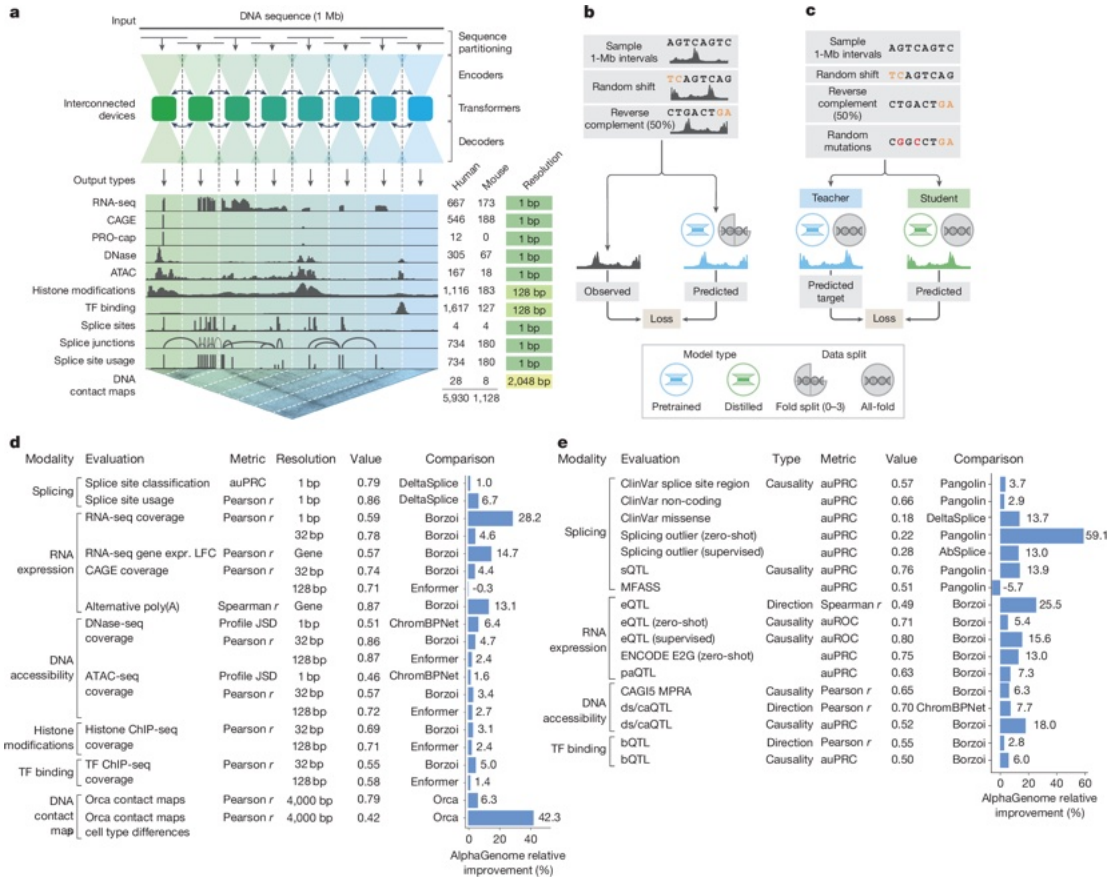
Several models fine-tuning Enformer for variant effect prediction, such as Performer

a Enformer 2.0: Borzoi



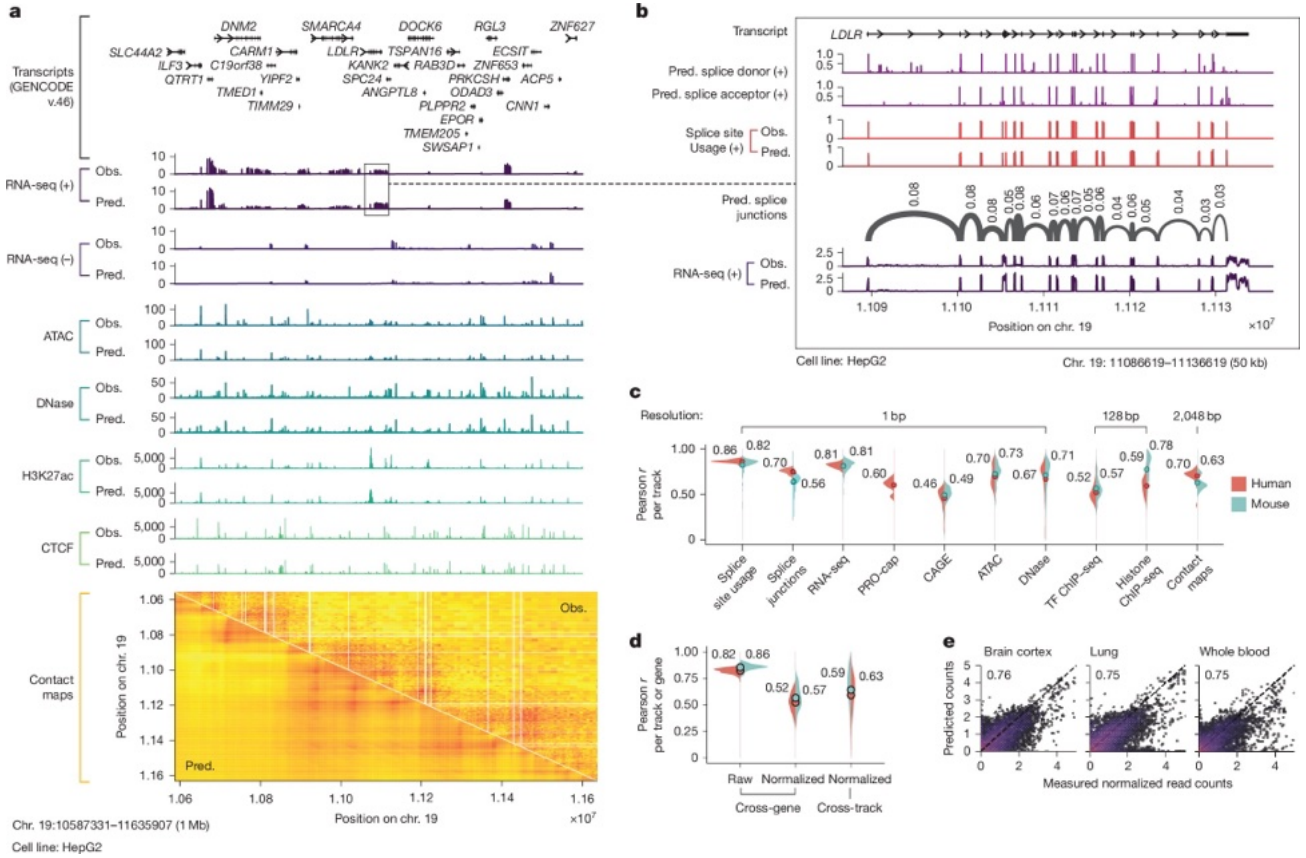






Advancing regulatory variant effect prediction with AlphaGenome

Avsec, Ž. *et al* (2026) Nature. https://doi.org/10.1038/s41586-025-10014-0



Advancing regulatory variant effect prediction with AlphaGenome

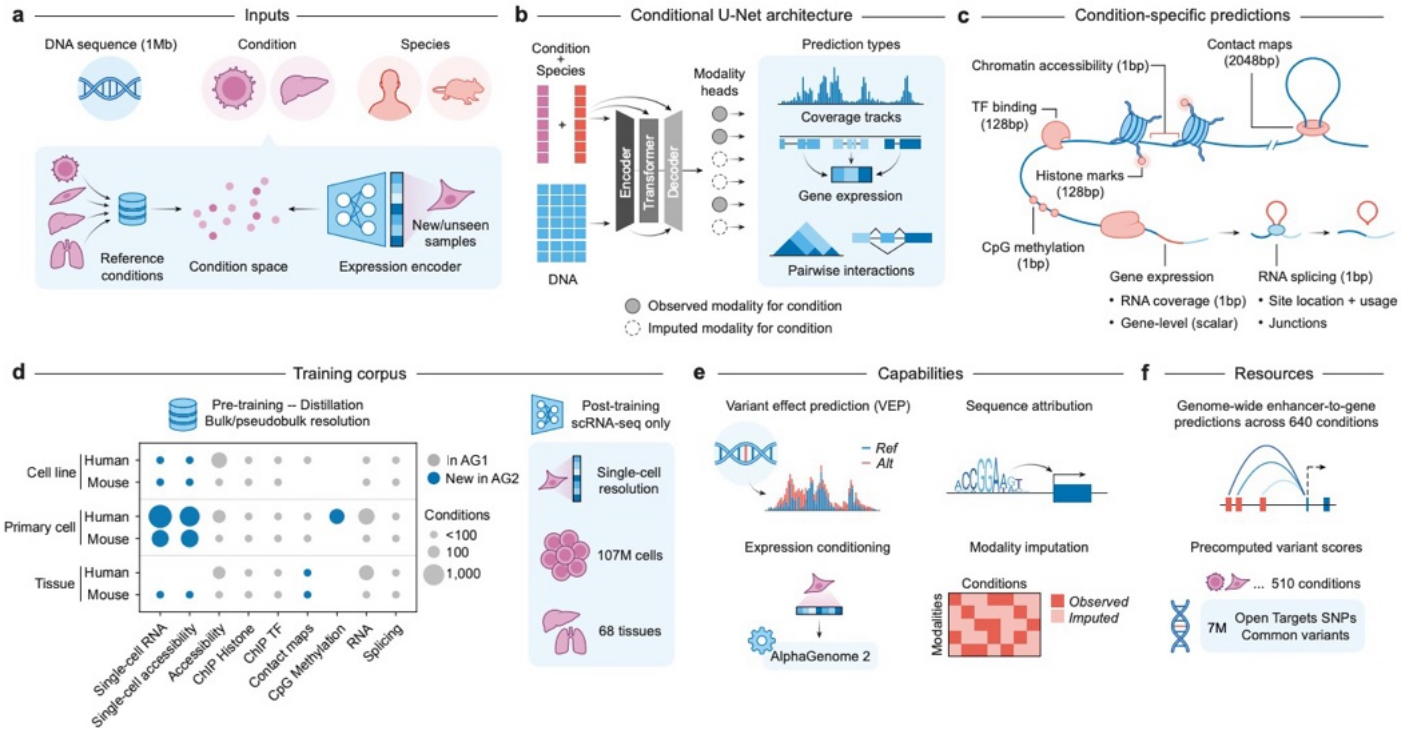
Avsec, Ž. *et al* (2026) Nature. https://doi.org/10.1038/s41586-025-10014-0

AlphaGenome 2 predicts regulatory activity and variant effects across cell states

Adam Gayoso[1,*,✉], Anne Mottram[1,*], Alexander Karollus[1,*], Guido Novati[1,*], Pavol Drotár[1,*], John Reid[1,*], Laetitia Meng-Papaxanthos[1,*], Adam Kosiorek[1,*], Eirini Arvaniti[1,*], Vincent Dutordoir[1], Soham De[1], Tom Ward[1], Lai Hong Wong[1], Matteo Perino[1], Kyle Taylor[1], Jun Cheng[1], Natasha Latysheva[1], Joshua Pan[1], Dhavanthi Hariharan[1], Clare Bycroft[1], Lauren Nicolaisen[1], Demis Hassabis[1], Pushmeet Kohli[1,✉] and Žiga Avsec[1,✉]

[1]Google DeepMind
[*]Equal contribution
[✉]Corresponding authors: adamgayoso@google.com, pushmeet-contact@google.com, avsec@google.com

Most genetic variants associated with human traits and diseases li
alter gene regulation in specific cell types and states. Single-cell atl
and development, but rarely profile enough donors to associate va
learning models could bridge this gap by predicting variant effects
distinguish related cell states and require fine-tuning for new datas
(AG2), which substantially advances cell-type-specific prediction of
for gene expression and chromatin accessibility. Trained on organis
atlases and bulk functional genomics data, AG2 takes as input a 1
specifying a target tissue, cell type, or cell state. Conditioning
epigenomic tracks and predict multimodal variant effects across cell
from gene expression extends AG2 to unseen datasets and cell states
fine-tuned AlphaGenome 1 and specialized models trained on targ
single-cell eQTLs at 90% precision. AG2 also achieves state-of-th
pinpoint causal variants at complex trait loci. Across hundreds of hu
of enhancer–gene links and scores for 7 million common variants. A
development, tissues, and disease cohorts, AG2 provides a blueprint
variant effect maps.



Evo2

- The largest genomic LLM created to date
- Published February 2025
- Very divisive in the field

Genome modeling and design across all domains of life with Evo 2

Garyk Brixi[*,1,2,3], Matthew G. Durrant[*,1,2], Jerome Ku[*,1,2], Michael Poli[*,2,3,5], Greg Brockman[**,2,6,§], Daniel Chang[**,1,2,3], Gabriel A. Gonzalez[**,1,2], Samuel H. King[**,1,2,3], David B. Li[**,1,2,3], Aditi T. Merchant[**,1,2,3], Mohsen Naghipourfar[**,1,2,7], Eric Nguyen[**,2,3], Chiara Ricci-Tam[**,1,2], David W. Romero[**,2,4], Gwanggyu Sun[**,1,2], Ali Taghibakshi[**,2,4], Anton Vorontsov[**,2,4], Brandon Yang[**,2,6], Myra Deng[8], Liv Gorton[8], Nam Nguyen[8], Nicholas K. Wang[8], Etowah Adams[9], Stephen A. Baccus[3], Steven Dillmann[3], Stefano Ermon[3], Daniel Guo[1,3], Rajesh Ilango[1], Ken Janik[4], Amy X. Lu[7], Reshma Mehta[6], Mohammad R.K. Mofrad[7], Madelena Y. Ng[3], Jaspreet Pannu[3], Christopher Ré[3], Jonathan C. Schmok[1], John St. John[4], Jeremy Sullivan[1], Kevin Zhu[7], Greg Zynda[4], Daniel Balsam[8,10], Patrick Collison[1,10], Anthony B. Costa[4,10], Tina Hernandez-Boussard[3,10], Eric Ho[8,10], Ming-Yu Liu[4,10], Thomas McGrath[8,10], Kimberly Powell[4,10], Dave P. Burke[‡,1,2,10], Hani Goodarzi[‡,1,2,10,11], Patrick D. Hsu[‡,†,1,2,7,10], Brian L. Hie[‡,†,1,2,3,10]

[1]Arc Institute; [2]Core Contributor, Evo 2 Team; [3]Stanford University; [4]NVIDIA; [5]Liquid AI; [6]Independent Researcher; [7]University of California, Berkeley; [8]Goodfire; [9]Columbia University; [10]Senior Contributor, Evo 2 Team; [11]University of California, San Francisco

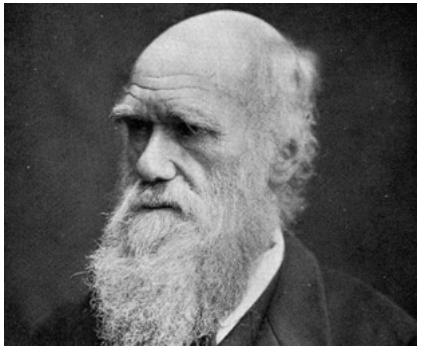
Extreme training requirements

- “Evo 2 was trained for **several months** on the NVIDIA DGX Cloud AI platform via AWS, utilizing over **2,000** NVIDIA **H100 GPUs** and bolstered by **collaboration with NVIDIA** researchers and engineers.” - Arc Institute blog post
- Roughly **$10M** training

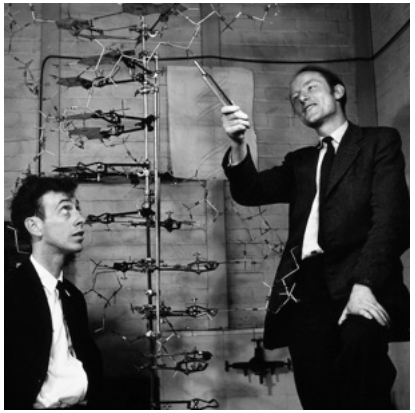


Braggadocious framing

- “Biological research scales from molecules to systems to organisms, seeking to understand and design functional components across all domains of life (**Darwin**, 1859; **Mendel**, 1866; Dobzhansky, 1951) … All domains of life express complex functions from DNA sequences (**Watson and Crick**, 1953; Nirenberg and Matthaei, 1961), yet genomic content and length vary dramatically across organisms.”





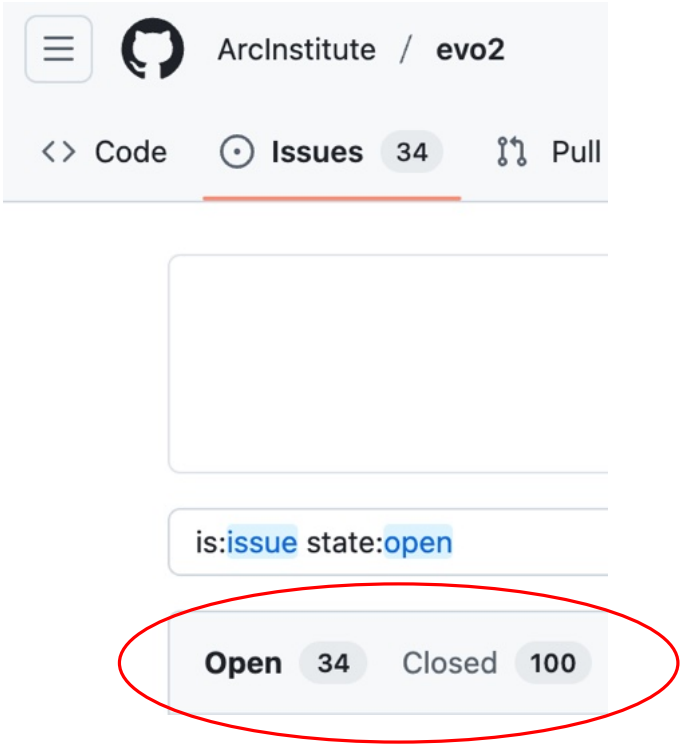


Difficulty of use

- Evo2-40B weights take up **80GB** in GPU RAM
- H100 node required to compile the code and run inference
- Can take **hours** of inference time to process a **single sample**

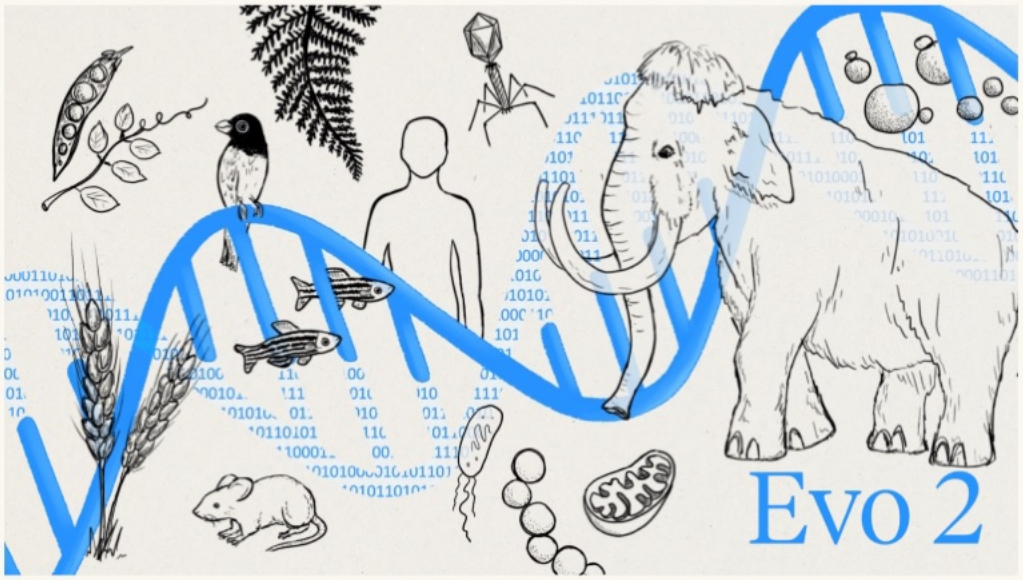


Nvidia H100 NVL GPU Computing Processor
$29,500.00
Pre-owned
eBay & more
4.0 ★★★★☆ (1)



What makes Evo2 so impressive?

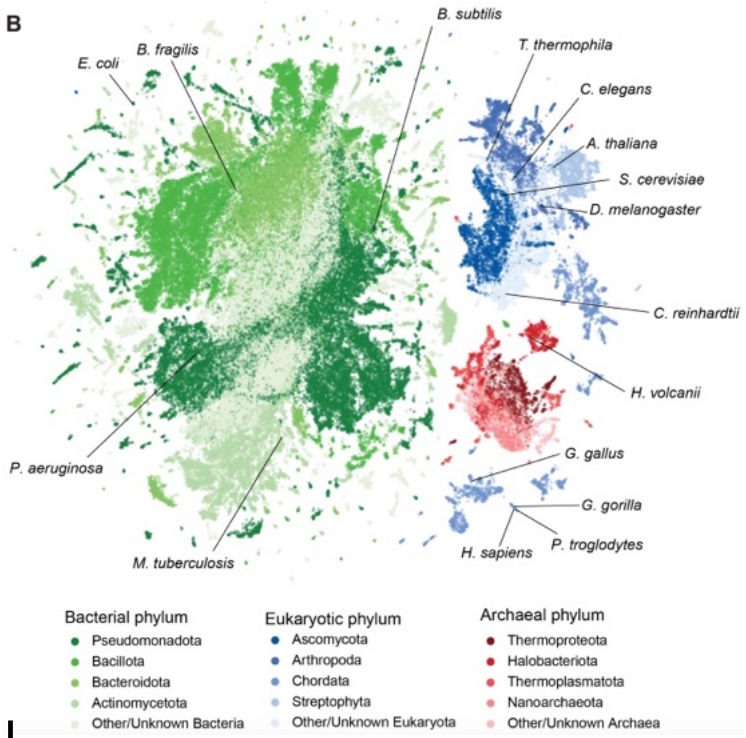
Arc Institute develops the largest AI model for biology to date in collaboration with NVIDIA, bringing together Stanford University, UC Berkeley, and UC San Francisco researchers



Evo 2 is trained on over 9.3 trillion tokens–in this case, nucleotides–from over 128 thousand genomes across the three domains of life, making it similar in scale to the most powerful generative AI large language models.

Training data

- OpenGenome2 dataset
- 113k prokaryotic genomes (357B bp)
- 15,032 eukaryotic genomes (6.98T bp)
- Non-redundant metagenomes (854B bp)
- Euk. organelle genomes (2.82B bp)
- Euk. coding sequences (602B bp)
- 8.8T bases total
- Data has no labels; **self-supervised** model

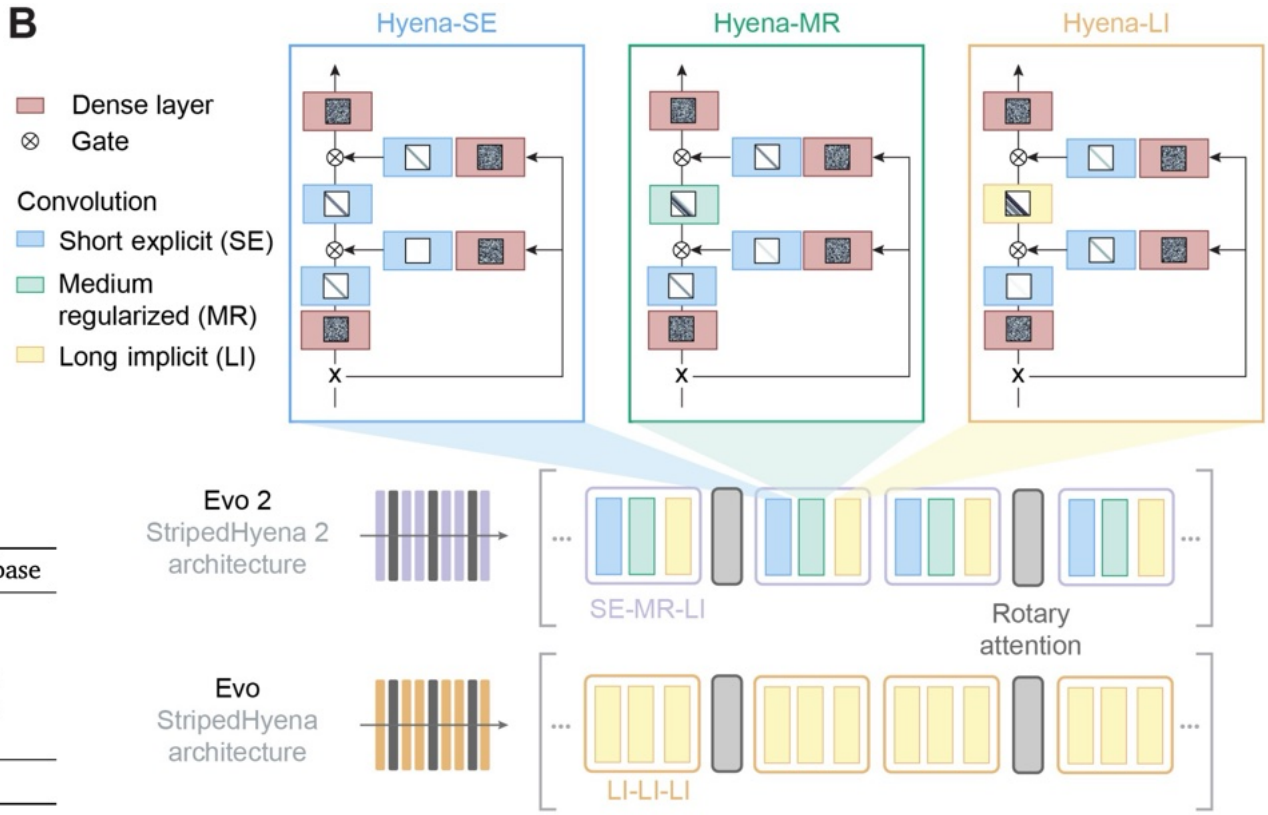


Model architecture

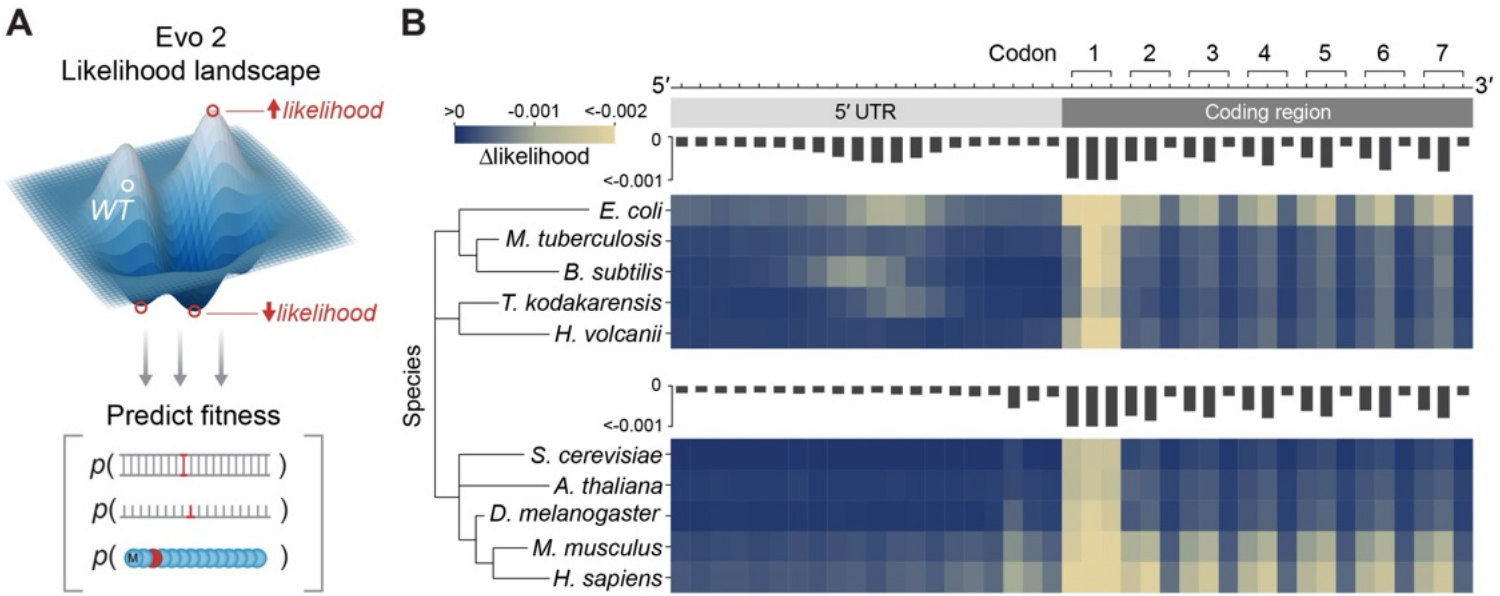
- **Convolutions** find **features**
- **Attention** gives features **context**
- **Early** layer features are **simple**, like kmer motifs
- **Late** layer features are **complex**, like gene-specific CREs

- StripedHyena 2
- Intentionally **patterned** series of **convolutions** and **attention** layers
- Mix of 7, 128, and full-length convolutions

	Evo 2 40B	Evo 2 7B	Evo 2 1B base
Parameters	40.3B	6.5B	1.1B
Total Layers	50	32	25
Hidden Size	8,192	4,096	1,920
FFN Size	22,528	11,264	5,120
Num Heads	64	32	15
Total Tokens	9.3T	2.4T	1T

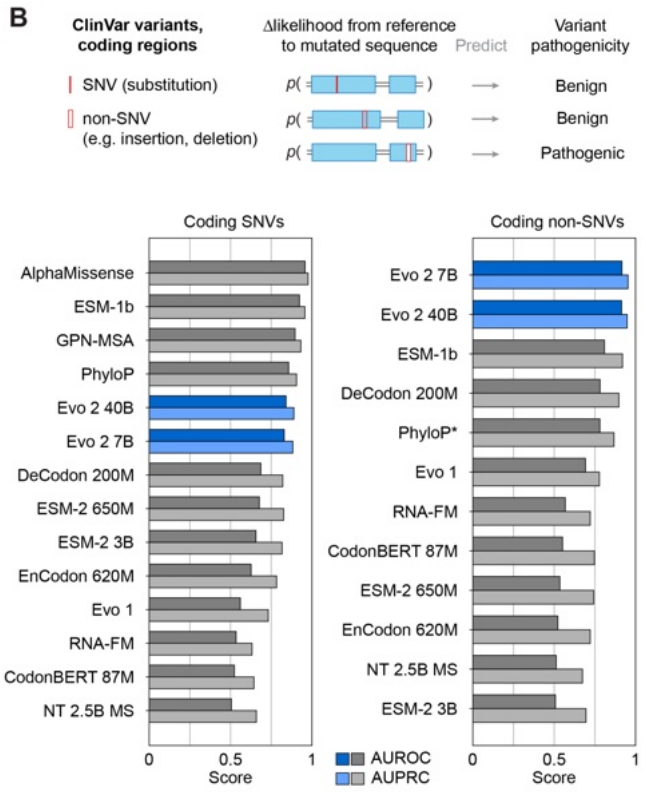


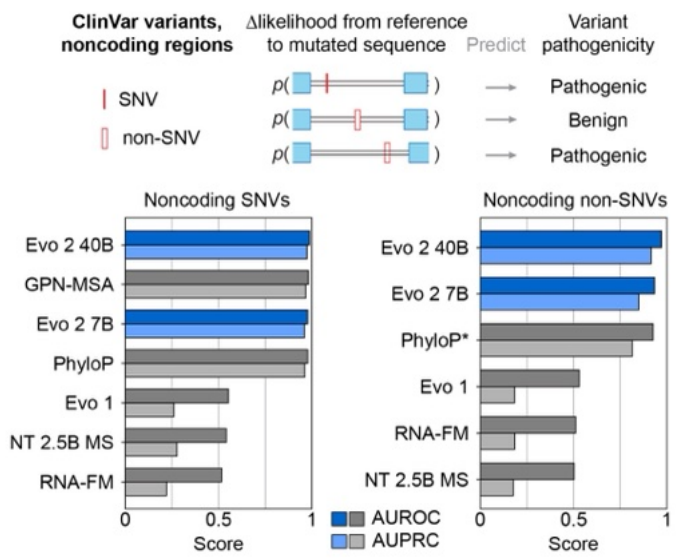
Evo2 learns coding syntax without any labels



- **Mutate bases** at different positions and measure **change** in sequence **likelihood**
- Learns that mutations in the **1st codon base** are **very impactful** / unlikely
- Learns that mutations in the **3rd codon base** (wobble position) are **less impactful**

Good Evo2 zero-shot performance on ClinVar

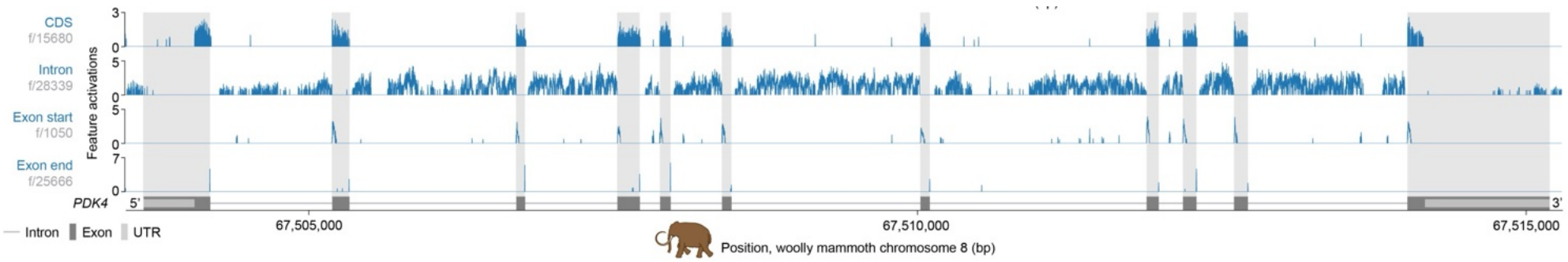




- Uses **sequence likelihood** as a proxy for **variant effect**
  - Loss-of-function variants tend to be pathogenic
- 15k coding, 38k noncoding variants
- Competes with protein LMs and **exceeds** DNA LMs

Interpretable features can annotate genomes

- Use four features to annotate the woolly mammoth genome
- Introns, exons, and coding regions



Enformer vs. Evo 2

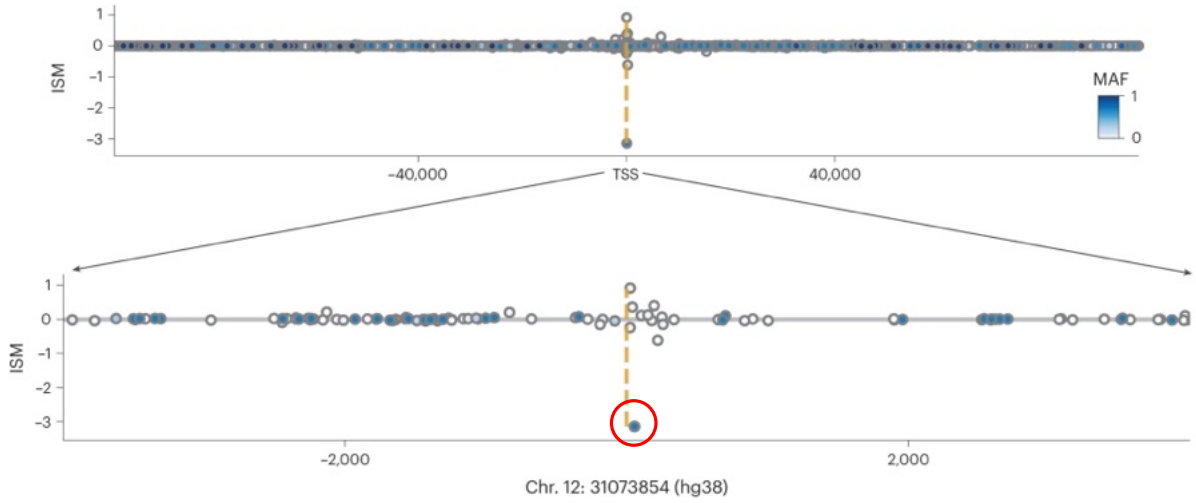
- Represent **two paradigms** in sequence-to-function modelling
- Both have inspired a wide range of follow-up studies

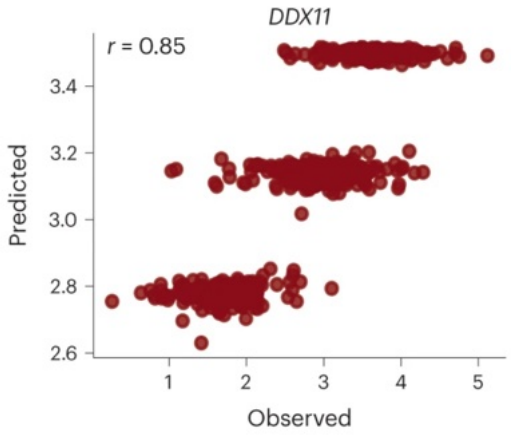
	Enformer	Evo 2
Training strategy	Supervised	Self-supervised
Training genomes	Human + mouse	Thousands of organisms
Bases of training sequence	6 billion	9.3 trillion
Input context window	197 Kbp	1 Mbp
Parameters	250 M	40 B
Text processing module	Transformer	Striped Hyena 2

Sasse et al. *Nature Genetics* (2023).

Using LLMs for variant eQTLs

- Predict **expression** directly from **sequence**
- In theory, can resolve linkage disequilibrium

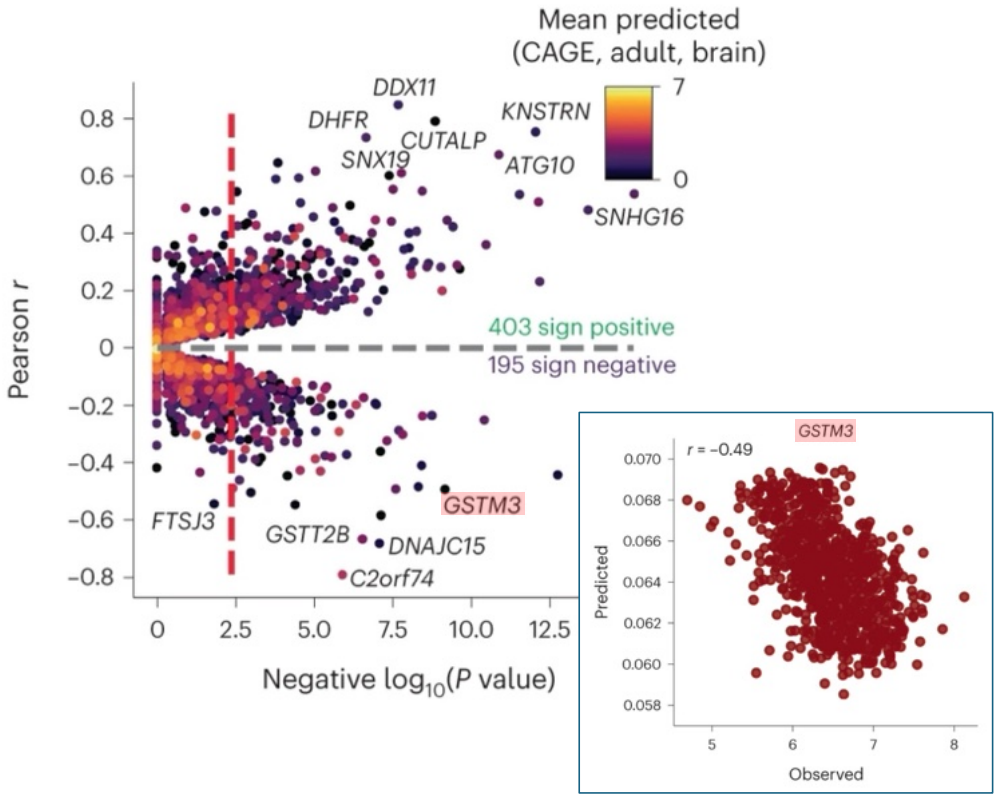




- Enformer can use a **single CNV** out of many candidates to predict eQTL
- Selects same CNV identified by fine-mapping with Susie

Sasse et al. *Nature Genetics* (2023).

LLMs do not truly understand variants



- Previous slide is the **exception**, not the rule
- Overall, LLMs know **if** gene expression changes from a mutation but not **how** it changes
- Flip a coin as to whether expression goes up or down

LLMs do not truly understand variants pt. 2

- Enformer is trained on the **reference** genome, not any **individual** sequences
- Top left: Enformer predicts population-average gene expression
- Bottom left: Enformer being right depends on finding causal SNVs
- Top,bottom right: linear model PrediXcan (purple) outperforms Enformer on individual-level predictions

